CIVICCONNECT

Project Engineering Document (PED) v3.0
Controlled Construction, Integration, Quality & Release Readiness

Study	SEN381 — Software Engineering 381
Milestone	3 — Integrated release candidate and evidence index
Team lead	Thaban Ndlozi — 602341
Team member	Anele Nkayi — 577168
Team member	Lebogang Masia — 601562
Version	3.0, continued from PED v2.0
M3 preparation date	12 October 2026; human approval/submission pending
Existing GitHub URL	PENDING — perform after project-ready review
Assessed RC tag/commit	PENDING — placeholder m3-rc1-pending-git-tag is not a tag
Continued RTM location	docs/registers/RTM_CONTINUED.csv (reconcile with existing live RTM)
CI / test evidence	.github/workflows/ci.yml; evidence/; actual CI run pending
Staging/demo	http://localhost:8081 (local simulated staging; not deployed yet)
Submission date	PENDING

Document continuation and evidence rule

The approved PED v2.0 source pages appear next **unchanged**, preserving the M1/M2 requirements, RTM, risks, decisions and sign-offs. Part D follows them with the M3 changes and an authentic evidence index. This is one continuing PED, not a second independent milestone report. Any tests, approvals, live CI or deployment marked pending remain pending until the team performs and records them.

History: PED v1.0 (9 Sep 2026) → PED v2.0 (26 Sep 2026) → PED v3.0 (12 Oct 2026 draft).
Status: NOT READY — runtime/CI/PR/evidence and stakeholder acceptance pending.

SOFTWARE ENGINEERING 381 - SEN381

CIVICCONNECT

PROJECT ENGINEERING DOCUMENT (PED)

Milestone 2 - Architecture, Technology & Initial Design Baseline

Project Engineering Document v2.0

Item	Details
Project	CivicConnect - Community Service Request Management Platform
Team lead	Thaban Ndlozi - 602341
Team member	Anele Nkayi - 577168
Team member	Lebogang Masia - 601562
Version	2.0
Date	26 September 2026
Github repository link	https://github.com/Thabani-Ndlozi/SEN381-CivicConnect

Document Control

Version	Date	Status / change	Review / approval
1.0	9 September 2026	Approved Milestone 1 Engineering Foundation & Requirements Baseline.	Approved by Thaban Ndlozi, Anele Nkayi and Lebogang Masia.
2.0	26 September 2026	Milestone 2 Architecture, Technology & Initial Design Baseline.	Current integrated PED v2.0.

Document Structure

- Part A - Approved Milestone 1 Baseline (PED v1.0)
- Part B - Milestone 2 Evolution
- B1 - Anele Nkayi: ASRs, Architecture and Design Decision 1
- B2 - Lebogang Masia: Data, Persistence, Design Decision 2, Integration and RTM
- B3 - Thaban Ndlozi: Technology, Deployment, SCM/CI and Engineering Control
- Part C - M2 Baseline Review and Sign-off

PART A - APPROVED MILESTONE 1 BASELINE (PED v1.0)

The approved M1 baseline is carried forward into PED v2.0. M2 extends this baseline; it does not replace it. Material M1 changes must use controlled change.

1. Source Basis and Contribution Boundary

This section is based on the SEN381 CivicConnect Master Project Brief and the Milestone 1 brief. The Master Brief defines CivicConnect as a community service request management platform and describes the current fragmented request process, the minimum requester/staff/management capabilities, and the mandatory project constraints (Belgium Campus ITversity, 2026a).

No stakeholder interview evidence has been supplied at this stage. Therefore, stakeholder needs that come directly from the brief are treated as source-backed, while additional lifecycle stakeholders and influence/interest ratings are clearly identified as team analysis rather than interviewed fact.

2. Problem and Business Need

2.1 Problem Analysis

The organisation currently receives service requests through a fragmented combination of email, telephone calls, WhatsApp messages, spreadsheets and paper records. This creates recurring problems such as duplicate or lost requests, weak assignment and ownership, limited status visibility for requesters, inconsistent reporting, weak accountability for status/action changes, and inconsistent handling of sensitive information. The core engineering problem is therefore the absence of one controlled, traceable record of the service-request lifecycle rather than merely the absence of a new user interface (Belgium Campus ITversity, 2026a).

Observed problem	Engineering implication
Requests may be duplicated, overlooked, incorrectly assigned or lost.	CivicConnect must support controlled request capture and clear ownership.
Requesters have limited visibility of request progress.	Requester-facing status, history and meaningful feedback are required business capabilities.
Staff struggle to prioritise, coordinate and identify ownership.	Authorised staff need useful request views, search/filter/sort and responsibility control.
Status/action accountability is weak.	The solution must preserve a controlled lifecycle and traceable actions.
Management information is unreliable or manual.	Structured request information must support oversight of open, overdue, resolved and closed work.
Sensitive information may be handled inconsistently.	Security and authorised access must be considered from the beginning of the lifecycle.

2.2 Business Need and Intended Value

The business need is for a controlled digital platform that provides a reliable, traceable and usable way to submit, manage, monitor and report service requests, while avoiding an unsustainable technical, operational or financial burden (Belgium Campus ITversity, 2026a).

Value area	Expected value
Visibility	Requesters can determine whether their requests were received and what the current state is.
Accountability	Responsibility, actions and lifecycle changes are easier to trace.
Operational control	Staff can identify relevant work and ownership using consistent request information.
Management oversight	Open, overdue, resolved and closed requests can be identified from controlled records.
Information quality	A single controlled lifecycle record reduces fragmented and inconsistent reporting.
Sustainability	Scope and later technology decisions remain constrained by team capacity, cost, quality, security and operational needs.

3. Stakeholder Analysis

The Master Brief explicitly describes three primary user groups: requesters, authorised staff, and management/oversight (Belgium Campus ITversity, 2026a). The analysis below also identifies project/lifecycle stakeholders that are implied by the organisation's business need and by mandatory operational/governance responsibilities. Those inferred stakeholders are labelled as such. Influence and interest ratings are team assessments for M1, not facts supplied by an interview.

ID	Stakeholder	Basis	Needs / expectations	Influence / interest	Main conflict / trade-off
STK-01	Requester	Explicit primary product stakeholder	Submit requests with appropriate information; categorise requests; view current status and history; receive meaningful feedback; have own information handled appropriately.	High / High	Transparency must not expose sensitive information to unauthorised people.
STK-02	Authorised service staff	Explicit primary product stakeholder	View relevant requests; search/filter/sort; view details; take/assign responsibility; update status through controlled transitions; record actions/comments; resolve/close where authorised.	High / High	Operational flexibility can conflict with lifecycle control and accountability.
STK-03	Management / oversight	Explicit primary product stakeholder	View useful service activity information; identify open/overdue/resolved/closed work; analyse by category/status or justified dimensions; support accountability and service-performance analysis.	High / High	Demand for broader reporting can expand scope and complexity.
STK-04	Organisation / service owner	Inferred business stakeholder	A sustainable controlled platform that improves visibility and accountability without excessive technical, operational or financial burden.	High / High	Low-cost expectations can conflict with resilience, security, monitoring or advanced functionality.
STK-05	System administration / operations	Inferred lifecycle stakeholder	Controlled configuration, access, secrets handling, deployment, logs, recovery and maintainable operation.	Medium / High	Operational controls consume effort that could otherwise be spent on visible features.
STK-06	SEN381 project team	Project delivery / engineering stakeholder	Clear scope, traceable requirements, manageable workload, controlled decisions, meaningful peer review and sufficient evidence to engineer and defend CivicConnect successfully.	High / High	Engineering governance and evidence activities consume schedule capacity that could otherwise be spent on implementation, but these controls are mandatory for project success.

The ratings above represent the team's M1 assessment rather than stakeholder interview evidence. Influence reflects the stakeholder's ability to affect product requirements, approval, operational use or project delivery, while interest reflects the extent to which CivicConnect's outcomes affect that stakeholder. These ratings will be reviewed if further stakeholder evidence becomes available.

3.1 Stakeholder Conflicts and M1 Responses

Conflict	Why it matters	M1 response
Requester transparency vs confidentiality	Requesters need meaningful status/history, but sensitive details must not be exposed to other requesters or unauthorised staff.	Baseline authorised access and controlled user-facing feedback; detailed identity design remains a later decision.
Staff flexibility vs lifecycle control	Staff must progress work efficiently, but unrestricted status changes would recreate weak accountability.	Require controlled status transitions and traceable actions; the exact transition catalogue remains an open business rule.
Management reporting vs scope discipline	Management may want many dashboards and reporting dimensions.	Commit only to the minimum oversight capabilities in the brief; advanced analytics/export remains deferred unless justified.
Low cost vs operational readiness	Free/low-cost services are preferred, but security, recovery and observability still matter.	Treat cost as a decision criterion, not permission to ignore quality/security/operations.
Feature breadth vs quality/evidence	Additional features consume time needed for requirements, reviews, testing, security and evidence.	Use a conservative scope baseline and assess additions through controlled change.

4. Scope Baseline

The Master Brief provides minimum business capabilities but states that these are not complete detailed requirements. The scope below converts those capabilities into a controlled M1 boundary and deliberately avoids committing the team to additional integrations or technology choices before evidence exists (Belgium Campus ITversity, 2026a; Belgium Campus ITversity, 2026b).

4.1 In Scope

- Direct submission of service requests into CivicConnect with appropriate request information.
- Selection of a request category through a controlled category mechanism.
- Requester access to the current status and history/list of previously submitted requests.
- Meaningful requester feedback when a request is accepted, rejected, updated or completed.
- Authorised staff access to relevant service requests, including search, filtering, sorting and full request details.

- Assignment or acceptance of responsibility for a request.
- Controlled request status updates and recording of relevant actions, comments or resolution information.
- Authorised resolution and closure of service requests.
- Management/oversight views that identify open, overdue, resolved and closed work and support category/status analysis.

4.2 Out of Scope for the Initial Baseline

- Automatic ingestion or synchronisation of requests from email, telephone calls, WhatsApp, spreadsheets or paper records. These channels describe the current problem, but direct integration is not a minimum capability.
- Payments, inventory, asset-management or unrelated business modules outside the service-request problem.
- AI chatbot or autonomous AI classification/decision-making. These are not minimum capabilities and would create additional accuracy, privacy, security and verification obligations.

4.3 Deferred / Future Scope

- External email/SMS/WhatsApp notifications, subject to later cost, privacy, integration and operational evidence.
- File/photo attachments, subject to storage, privacy, malware-scanning and cost analysis.
- Advanced analytics, data exports, automated escalation and advanced service-level management are deferred until detailed management rules and stakeholder value are validated. A basic rule for determining whether a request is overdue remains within the baseline because overdue request identification is a minimum management capability.
- Third-party integrations and single sign-on after organisational systems and technology/environment constraints are known.
- Delivery platform implementation, including whether CivicConnect will use a web-based, native mobile or other client interface, is deferred until evidence-based technology and interface decisions are made in a later milestone.

4.4 Deliberate Exclusion to Defend

Automatic legacy-channel integration is deliberately excluded from the initial baseline.

The current fragmented process uses email, telephone, WhatsApp, spreadsheets and paper, but the minimum CivicConnect capabilities require the platform to support request submission and lifecycle management; they do not require automatic ingestion from those channels. Including such integrations now would introduce third-party dependencies, duplicate-handling rules, privacy/security concerns, additional testing and schedule risk. A direct CivicConnect submission path therefore provides the required business capability while keeping the initial scope controllable. The team may reconsider integrations later through formal impact analysis if stakeholder value justifies them.

5. Assumptions and Constraints

5.1 Explicit Project Constraints and Mandatory Engineering Controls

The following project constraints and mandatory engineering controls are derived directly from the Master Project Brief (Belgium Campus ITversity, 2026a). Their engineering implications represent the team's analysis of how these controls should influence later project work.

ID	Type	Constraint	Source	Engineering implication
C-001	Team size	Exactly 3 students.	Master Brief Section 4	All controlled work must fit a three-person team; substantive PR review capacity must be planned.
C-002	Schedule	Four formal milestones within the SEN381 delivery period.	Master Brief Section 4	Scope and evidence must progress incrementally; late reconstruction of evidence is not acceptable.
C-003	Cost / resources	Prefer free or low-cost services where practical and identify limits/likely operational cost.	Master Brief Section 4, Section 18	Later platform choices must consider total cost and free-tier limitations.
C-004	Scope	Committed scope must be baselined and controlled.	Master Brief Section 4, Section 14	Additional features must undergo impact analysis instead of being silently absorbed.
C-005	Quality	Quality attributes must be defined and later supported by measurable evidence.	Master Brief Section 4, Section 15	Requirements must include testable quality expectations linked to later verification.
C-006	Security	Security is a lifecycle-wide responsibility, not a final add-on.	Master Brief Section 4, Section 16	Security/privacy concerns must influence requirements, repository controls, design, testing and deployment.
C-007	Technology	No stack, architecture or platform is prescribed.	Master Brief Section 4, Section 18.1	Technology selection must be deferred until evidence-based comparison against requirements, team capability, cost, security and deployment constraints.
C-008	Institutional environment	BC cannot guarantee support/compatibility for every technology or external service.	Master Brief Section 25	The team must investigate compatibility before committing and manage the risk of unavailable/unsupported technology.

ID	Type	Constraint	Source	Engineering implication
C-009	Configuration management	Protected main, PRs and two non-author approvals are mandatory for substantive changes.	Master Brief Section 9	Review capacity becomes part of the schedule; substantive direct development on main is not acceptable.
C-010	AI accountability	AI output is not authoritative evidence; material use must be recorded and verified.	Master Brief Section 10	AI-assisted work must be independently checked and remains the team's responsibility.

5.2 Controlled Assumptions / Evidence Gaps

The following assumptions identify information that is not specified in the project brief and will require validation as further project evidence becomes available.

ID	Assumption / evidence gap	Treatment
A-001	The supplied project briefs are the current authoritative source for M1; no stakeholder interview evidence has been supplied at this stage.	Validate if the lecturer/client provides additional evidence.
A-002	CivicConnect is treated as a single-organisation platform; multi-tenant operation is not baselined.	Revisit only if a stakeholder requires multi-tenancy.
A-003	Users will have controlled identities/roles, but the identity mechanism is not chosen in M1.	Require evidence before the M2 authentication decision.
A-004	The brief does not provide production workload volumes.	Do not claim workload/performance facts until evidence exists.
A-005	The exact category catalogue, status-transition catalogue and overdue rule are not supplied.	Treat as high-priority open business-rule gaps. The team must define and validate these rules before detailed design; however, the requirement to identify overdue requests remains baselined.
A-006	The brief states that sensitive request information may exist but does not define detailed data-classification or retention rules.	Perform security/privacy analysis before persistence design.

5.3 Constraint Interactions / Ripple Effects

Interaction	Engineering consequence
Three-person team + two non-author approvals + fixed milestone schedule	Because every substantive PR needs review by both non-author members, review capacity is a real schedule constraint. Large late PRs would create a bottleneck. The team should therefore use smaller coherent changes and review progressively.
Low cost + quality/security/operational readiness	A free platform is not automatically acceptable. Future technology/platform decisions must still support secure configuration, testing, deployment, recovery and operational evidence.
Scope + schedule + quality	Adding optional features increases implementation and verification work. Under schedule pressure the correct response is controlled scope change, not silently reducing testing, security or evidence.

6. Requirements Analysis

The requirements below convert the CivicConnect minimum business capabilities, stakeholder needs and M1 constraints into a controlled requirements baseline. Each requirement has a stable identifier, source, priority and acceptance criterion so that it can be traced and verified as the project progresses (Belgium Campus ITversity, 2026a; Belgium Campus ITversity, 2026b).

6.1 Functional Requirements

ID	Source / Stakeholder	Priority	Requirement	Acceptance Criteria
FR-001	STK-01; Master Brief Section 3	High	CivicConnect shall allow a requester to submit a new service request with the required request information.	AC-FR-001: A request with valid mandatory information is saved with a unique request identifier and confirmation is shown; missing mandatory information prevents submission and produces validation feedback.
FR-002	STK-01; Master Brief Section 3	High	CivicConnect shall allow a requester to categorise a request using a controlled category mechanism.	AC-FR-002: A requester can select a category from the approved category catalogue and the selected category is stored with the request. The final catalogue remains subject to A-005 validation.
FR-003	STK-01; Master Brief Section 3	High	CivicConnect shall allow an authorised requester to view the current status and history/list of previously submitted requests.	AC-FR-003: An authorised requester can open their request list/history and see the current status for each submitted request.

ID	Source / Stakeholder	Priority	Requirement	Acceptance Criteria
FR-004	STK-01; Master Brief Section 3	High	CivicConnect shall provide meaningful requester feedback when a request is accepted, rejected, updated or completed.	AC-FR-004: When one of the defined lifecycle events occurs, the requester can view corresponding feedback that reflects the current request state.
FR-005	STK-02; Master Brief Section 3	High	CivicConnect shall allow authorised service staff to view relevant requests, search/filter/sort them using useful criteria, and view full request details.	AC-FR-005: Authorised staff can view relevant requests, open request details, and successfully search/filter/sort using the baselined criteria.
FR-006	STK-02; Master Brief Section 3	High	CivicConnect shall allow authorised staff to assign or accept responsibility for a request.	AC-FR-006: An authorised staff member can assign or accept responsibility and the current request owner is recorded and visible to authorised staff.
FR-007	STK-02; Master Brief Section 3; A-005	High	CivicConnect shall allow authorised staff to update request status through controlled transitions and record relevant actions, comments and resolution information.	AC-FR-007: Only transitions permitted by the approved status-transition catalogue are accepted; an allowed transition is recorded together with the related action/comment/resolution information.
FR-008	STK-02; Master Brief Section 3	High	CivicConnect shall allow authorised staff to resolve or close requests where permitted.	AC-FR-008: An authorised staff member can resolve/close an eligible request and the resulting lifecycle state and resolution information are recorded.
FR-009	STK-03; Master Brief Section 3	High	CivicConnect shall provide management with service-activity information that identifies open, overdue, resolved and closed requests and supports category/status analysis.	AC-FR-009: Management can identify open, overdue, resolved and closed requests and view request information grouped or filtered by category and status. The exact overdue rule remains subject to A-005 validation.
FR-010	STK-03; STK-04; Master Brief Sections 2-3	High	CivicConnect shall preserve enough request lifecycle information to support accountability and service-performance analysis.	AC-FR-010: For tested ownership, status and resolution changes, authorised management can identify the recorded action, actor and timestamp in the request history/audit information.

6.2 Non-Functional Requirements

ID	Quality Attribute / Source	Priority	Requirement	Acceptance Criteria
NFR-001	Usability; STK-01/02/03; C-005	High	The core CivicConnect requester and staff workflows shall be usable without requiring external training material for the defined acceptance scenarios.	AC-NFR-001: All defined core-task acceptance scenarios for request submission, status/history access and staff request handling can be completed using the interface and on-screen guidance without external training instructions.
NFR-002	Authorisation / privacy; C-006; A-006	High	CivicConnect shall restrict sensitive request information and privileged functions to authorised roles.	AC-NFR-002: All defined unauthorised-access test cases are denied, while authorised role test cases can access only the functions and information permitted to that role.
NFR-003	Secrets / configuration security; C-006; C-009	High	Credentials, API keys, tokens, private keys and other secrets shall not be committed to the source repository and shall be supplied through approved configuration mechanisms.	AC-NFR-003: Repository review finds no committed secrets/credentials, and any required secret is obtained through the approved environment/configuration mechanism in the test/deployment setup.
NFR-004	Auditability / accountability; STK-03; C-005	High	Important lifecycle changes shall be traceable for accountability.	AC-NFR-004: For every tested ownership, status and resolution change, the system records the actor, action and timestamp and makes the record available to an authorised user.

6.3 Requirement Baseline Notes

- The exact category catalogue, status-transition catalogue and overdue rule remain controlled business-rule gaps under A-005 and must be validated before detailed design.
- The identity/authentication mechanism is intentionally not selected in M1; the access-control requirements are baselined without prematurely selecting the implementation mechanism.
- No production workload volumes are available under A-004, so no unsupported production performance or scalability target is claimed in this baseline.
- Any change to a baselined requirement must be impact-analysed and reflected in the requirement, acceptance criteria, RTM, risks and affected downstream artefacts.

7. Initial Requirements Traceability Matrix

The RTM establishes the M1 traceability path from stakeholder/source to requirement and acceptance criteria, while reserving explicit fields for later design, implementation, test and release evidence (Belgium Campus ITversity, 2026b).

Source / Stakeholder	Requirement ID	Requirement	Priority	Acceptance Criteria	Issue / PR	Design / Architecture	Implementation	Test	Acceptance / Release Evidence
STK-01 / Master Section 3	FR-001	Submit service request	High	AC-FR-001	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-01 / Master Section 3	FR-002	Controlled categorisation	High	AC-FR-002	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-01 / Master Section 3	FR-003	View status and history	High	AC-FR-003	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-01 / Master Section 3	FR-004	Requester lifecycle feedback	High	AC-FR-004	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-02 / Master Section 3	FR-005	Staff view/search/filter/details	High	AC-FR-005	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-02 / Master Section 3	FR-006	Assign/accept responsibility	High	AC-FR-006	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-02 / A-005	FR-007	Controlled status/actions/comments	High	AC-FR-007	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-02 / Master Section 3	FR-008	Resolve/close requests	High	AC-FR-008	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-03 / Master Section 3	FR-009	Management activity oversight	High	AC-FR-009	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
STK-03/04 / Master Sections 2-3	FR-010	Lifecycle accountability information	High	AC-FR-010	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
C-005 / STK-01/02/03	NFR-001	Core workflow usability	High	AC-NFR-001	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
C-006 / A-006	NFR-002	Role-based access/privacy	High	AC-NFR-002	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4

Source / Stakeholder	Requirement ID	Requirement	Priority	Acceptance Criteria	Issue / PR	Design / Architecture	Implementation	Test	Acceptance / Release Evidence
C-006 / C-009	NFR-003	Secrets/configuration security	High	AC-NFR-003	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4
C-005 / STK-03	NFR-004	Auditability of lifecycle changes	High	AC-NFR-004	Controlled docs/requirements artefact and current M1 requirements/RTM PR	Deferred to M2	Deferred to M3	Deferred to M3	Deferred to M4

The repository-hosted RTM is the live traceability record. Exact Issue/PR references should be kept current there as new controlled evidence is created.

8. Initial Risk Register

Team scoring convention: Probability 1-5 x Impact 1-5 = Risk Score. Priority bands: 1-4 Low, 5-9 Medium, 10-14 High, 15-25 Critical.

ID	Risk	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RISK-001	Requirements may be ambiguous or interpreted differently, causing rework or incorrect downstream decisions.	The brief contains broad capabilities and some business rules remain open.	3 - Possible	4 - Major	12 High	Use stable IDs, explicit sources, priorities and measurable acceptance criteria; team-review requirements and maintain the RTM.	Clarify the source; revise the requirement/AC through controlled change; update RTM and dependent artefacts.	L. Masia	Open
RISK-002	Scope may expand beyond the capacity of a three-person team and fixed milestone schedule.	Optional features/new requests may be accepted without adequate impact analysis.	4 - Likely	5 - Severe	20 Critical	Maintain explicit in/out/deferred scope and analyse schedule, resource, quality, security and risk impact before additions.	Defer lower-priority functionality; record the approved scope change and update requirements, RTM, risks and planning.	A. Nkayi	Open
RISK-003	Uneven availability or contribution may delay artefacts, reviews or milestone readiness.	Academic workload, timetable clashes, absence and two-reviewer approval requirements may create bottlenecks.	3 - Possible	4 - Major	12 High	Use clear issue ownership/internal deadlines, small progressive PRs and regular progress checks; raise blockers early.	Redistribute work by team agreement, update ownership/deadlines and escalate persistent participation issues where necessary.	T. Ndlozi	Open
RISK-004	Security/privacy requirements may be identified too late, causing redesign or inappropriate information handling.	Roles, permissions, data classification and detailed security rules are not fully defined in M1.	3 - Possible	5 - Severe	15 Critical	Capture security/privacy requirements early; identify role/access expectations and sensitive data; review security impact in requirements and PRs.	Raise controlled change; revise affected requirements/data-access expectations, update traceability/risks and reconsider later architecture decisions.	T. Ndlozi	Open
RISK-005	A later technology stack/platform may be incompatible, unsupported or too costly.	Stack/platform selection is deferred and environment, team-skill and operating-cost evidence is incomplete.	3 - Possible	4 - Major	12 High	Keep selection deferred in M1; in M2 compare viable options against requirements, capability, environment, security, deployment and cost evidence.	Select the best viable alternative using recorded decision criteria; update ADR/decision record, schedule implications and risks.	T. Ndlozi	Open
RISK-006	Important requirements may later be difficult or expensive to verify.	Acceptance criteria or NFRs may use vague/subjective wording.	3 - Possible	4 - Major	12 High	Use testable FRs and measurable NFRs with acceptance criteria; review wording for ambiguity/testability.	Refine before baseline or process through controlled change after baseline; update RTM and later verification plans.	L. Masia	Open
RISK-007	The team may apply GitHub governance inconsistently, weakening traceability and review evidence.	Members may be unfamiliar with the workflow or treat document review controls as unnecessary overhead.	2 - Unlikely	4 - Major	8 Medium	Protect main; use issues, branches, PRs, two non-author approvals, templates, meaningful commits and resolved review conversations.	Create corrective issue, restore the controlled process, re-review affected work and record the corrective action.	T. Ndlozi	Monitoring

ID	Risk	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RISK-008	AI-assisted work may introduce incorrect assumptions, unsupported claims or misleading content.	Generative AI can produce plausible but inaccurate output that may be accepted without sufficient verification.	3 - Possible	4 - Major	12 High	Record material AI use; verify important claims against source/project evidence; require human review; never expose secrets/sensitive information.	Reject/revise incorrect output; correct affected artefacts, update AI/traceability records and re-review before baseline/merge.	T. Ndlozi	Monitoring

9. Forward Engineering Considerations

These considerations identify later lifecycle concerns that already influence M1 requirements, constraints, risks or assumptions. They do not represent final architecture, technology, database, deployment or implementation decisions.

ID	Forward Engineering Consideration	Why it matters now	Later decision / activity affected	Information still missing	Risk if ignored
FEC-001	Security, Authentication & Role-Based Access	Requester, staff and management have different capabilities, so access expectations already affect requirements and acceptance criteria.	Authentication, authorisation, architecture, API access control, security testing and deployment configuration.	Exact roles, permission boundaries, authentication mechanism, sensitive-data classification and detailed security requirements.	Inappropriate access may be enabled, causing redesign or security weaknesses.
FEC-002	Testability & Verification Strategy	M1 FRs, NFRs and acceptance criteria need to be verifiable later.	Test strategy, automation, integration/system/performance testing and release quality gates.	Final quality thresholds, test environments/data, performance expectations and testing tools.	Important requirements may prove vague, expensive or difficult to verify.
FEC-003	Deployment Environment & Platform Compatibility	The team should not select a stack without evidence that it can be developed, hosted and operated in available environments.	Technology-stack selection, architecture, CI/CD, hosting, configuration and production deployment.	Supported runtimes/platforms, hosting options, environment limitations, service availability, free-tier limits and operating costs.	A preferred stack may later be unavailable, incompatible, difficult to deploy or too expensive.
FEC-004	Data Architecture, Audit History & Recovery	CivicConnect must store requests, statuses, ownership, comments/actions and enough history for accountability/reporting.	Database design, API contracts, validation, backup/recovery, reporting and migration.	Detailed entities, retention, audit-history rules, backup frequency, recovery expectations and likely volume.	Poor assumptions may cause missing audit data, schema rework, unreliable reporting or weak recovery.
FEC-005	Observability, Logging & Operational Support	A system that works in development must still be diagnosable and supportable when deployed.	Logging, monitoring, health checks, deployment design, incident response and operational support.	Hosting platform, monitoring tools, support responsibilities, alerting and availability expectations.	Failures may be discovered only after users complain and incidents may be difficult to diagnose.
FEC-006	Maintainability, Changeability & Technical Debt	Requirements/workflows may evolve as stakeholder understanding improves or changes are requested.	Architecture, modularity, technology selection, API/data design, testing and maintenance.	Likely future changes, support expectations, architectural constraints and final technology choices.	Short-term choices may create unnecessary coupling/technical debt and make later changes expensive.

9.1 Initial Deployment and Operational Considerations

- No deployment platform is selected in M1. Later selection must be justified against requirements, team capability, environment compatibility, security, likely operational cost and vendor/platform limitations.
- Development, test, staging and production environments must be distinguished later; success in a development environment will not be treated as production-readiness evidence.
- Environment-specific configuration and secrets, deployment/release approval, rollback/recovery, logging, monitoring, health checks and failure detection must be considered before release.
- The deployment decision must revisit A-004 workload uncertainty, A-006 sensitive-data gaps, C-003 cost/resource limits and C-008 institutional environment compatibility.

10. Engineering Decision Log

Only genuine M1 decisions and deliberate deferments are recorded. Decisions that require architecture, technology or implementation evidence are intentionally deferred rather than guessed.

ID	Context	Constraints	Alternatives	Decision / Deferment	Rationale	Trade-offs	Risks	Evidence	Later consequence
DEC-001	Legacy request channels are part of the current problem context.	Scope, schedule, security/privacy, three-person capacity.	Automatically integrate email/phone/WhatsApp/spreadsheets/paper; or require direct CivicConnect submission.	Exclude automatic legacy-channel ingestion from the initial baseline.	Direct submission satisfies the minimum capability without introducing unnecessary integrations.	Less convenience for legacy-channel users; lower integration/testing complexity.	Future stakeholder demand may require integration.	Section 4.4; STK-01/02; C-004.	Any later integration requires formal impact analysis and traceability updates.
DEC-002	The brief prescribes no stack, architecture or delivery platform.	C-003 cost, C-007 technology freedom, C-008 environment, A-004 workload evidence gap.	Select a familiar stack now; select a platform now; or defer until evidence is available.	Defer final architecture, technology stack and delivery-platform choice to M2.	M1 lacks sufficient evidence for a defensible consequential technology decision.	Preserves options but postpones implementation certainty.	Late discovery of incompatibility/learning-curve risk.	Sections 4.3, 5.1; RISK-005; FEC-003.	M2 must compare viable alternatives using evidence and record the final decision/ADR.
DEC-003	Management must identify overdue requests, but advanced SLA/escalation rules are not defined.	Minimum management capability, A-005 open overdue rule, schedule/scope control.	Defer all overdue logic; implement advanced SLA/escalation now; or baseline only basic overdue identification.	Keep basic overdue identification in scope and defer advanced SLA/escalation management.	The brief explicitly requires overdue identification, while detailed escalation/SLA rules lack evidence.	A basic overdue rule still requires validation; avoids premature advanced workflow.	Poorly defined rule could create incorrect overdue classification.	Section 4.3; FR-009; A-005.	Validate the overdue rule before detailed design and update FR-009/RTM if it changes.
DEC-004	Several optional capabilities may add value but are not minimum M1 commitments.	Scope, schedule, cost, security, risk and three-person capacity.	Include notifications, attachments, advanced analytics, SSO/integrations now; or defer.	Defer external notifications, attachments, advanced analytics/exports, SSO and third-party integrations.	A smaller controlled baseline provides stronger engineering evidence than broad unfinished scope.	Optional user convenience is postponed; verification burden is reduced.	Deferred features may later become stakeholder priorities.	Section 4.3; RISK-002; C-004.	Any addition must pass impact analysis and update scope, requirements, RTM and risks.

11. GitHub and Team Governance

GitHub is used as the controlled engineering evidence environment for CivicConnect. The controls below reflect the repository governance established for M1 and the mandatory Master Project Brief requirements (Belgium Campus ITversity, 2026a).

Control	M1 baseline	Controlled evidence
Repository link	One controlled CivicConnect team repository.	https://github.com/Thabani-Ndlozi/SEN381-CivicConnect
Main branch	Protected; substantive direct development on main is not permitted.	Repository ruleset / branch protection.
Work initiation	Meaningful substantive work is represented by an issue/task and performed on a task branch from current main.	.github/ISSUE_TEMPLATE/engineering-task.md; repository Issues.
Pull Requests	Substantive changes enter main through a Pull Request linked to the relevant issue where practical.	.github/pull_request_template.md; PR history.
Approvals	Two approvals are required from the two team members other than the author; self-approval is not accepted.	Ruleset and PR review history.
Review quality	Reviews must be meaningful; comments are addressed and conversations resolved before merge.	Review comments, corrections, re-reviews and approvals.
History	Commits, PRs, reviews and merges are progressive and authentic; bulk reconstruction is avoided.	Repository commit/PR history.
Secrets	Passwords, API keys, tokens, private keys and confidential credentials must not be committed.	Repository review and configuration practice.
Team Working Agreement	Task ownership, communication, review, conflict/delay handling, AI accountability and baseline approval are controlled in the team agreement.	docs/governance/team-working-agreement.md.

12. AI Usage Register

The live AI Usage Register is maintained as a controlled repository artefact at docs/governance/ai-usage-register.md. It is not duplicated in full inside the PED so that one current source of AI-accountability evidence is maintained.

At baseline review, the team must confirm that every material AI-assisted contribution is recorded with: Date, Student, Tool, Engineering task, AI contribution, Verification, Decision, and Issues found. AI output is not treated as authoritative evidence; the responsible student remains accountable for verification, understanding and defence of the result (Belgium Campus ITversity, 2026a; Belgium Campus ITversity, 2026b).

- The register must include the material AI assistance used for requirements/RTM work, stakeholder/scope/constraints work and any governance or other engineering artefact where AI materially contributed.
- Verification must describe what the student checked, changed, rejected or accepted rather than merely stating that AI was used.
- No credentials, secrets or inappropriate sensitive information may be supplied to AI tools.

13. Baseline Review and Sign-off

13.1 Version History

Version	Date	Status / change	Review / approval
0.1	8 September 2026	Initial integrated M1 working document.	In progress
0.9	9 September 2026	Complete M1 baseline candidate with requirements, RTM, risk, forward considerations, decision log, governance and sign-off structure integrated.	Ready for team baseline review
1.0	9 September 2026	Approved M1 baseline after team quality review.	All three members approved/signed.

13.2 Baseline Readiness Check

Check	Status at candidate review
Problem, business need, stakeholder analysis, scope and constraints reviewed	Approved - M1 baseline v1.0
Functional/NFR requirements and acceptance criteria reviewed for clarity, priority, source and testability	Approved - M1 baseline v1.0
RTM checked for source -> requirement -> acceptance criteria and future evidence structure	Approved - M1 baseline v1.0
Risk Register and Forward Engineering Considerations reviewed	Approved - M1 baseline v1.0

Check	Status at candidate review
Engineering Decision Log reviewed; deferred decisions have clear evidence needs	Approved - M1 baseline v1.0
GitHub governance and Team Working Agreement evidence checked	Approved - M1 baseline v1.0
AI Usage Register current and material AI use verified	Approved - M1 baseline v1.0
PED integrated, coherent and ready to baseline	Approved - M1 baseline v1.0

13.3 Team Sign-off

Baseline type: Milestone 1 - Engineering Foundation & Requirements Baseline. Target baseline version: PED v1.0.

Team member	Student number	Signature / approval	Date
Thaban Ndlozi	602341	T.Ndlozi	09/09/2026
Anele Nkayi	577168	A.Nkayi	09/09/2026
Lebogang Masia	601562	L. Masia	09/09/2026

Gate outcome: ACCEPTED - PED v1.0 approved by all three team members on 9 September 2026.

PART B - MILESTONE 2 EVOLUTION

M2 adds controlled architecture, data, technology, design, integration and development evidence to the approved M1 baseline.

B1 - Anele Nkayi: ASRs, Architecture and Design Decision 1

1. Purpose, Evidence Basis and Alignment

This section defines the M2 architecture baseline for CivicConnect: ASRs and quality drivers, architecture alternatives, the selected architecture, architecture diagrams, ADR-M2-ARCH-001 and one final design decision informed by Assignment 2. The architecture is part of PED v2.0 and links to the live RTM through the requirement, ASR, architecture and design fields (Belgium Campus ITversity, 2026c).

The work starts from the approved M1 baseline. M1 already defined the users, scope, constraints, functional requirements, security requirements, auditability requirement, risks and forward engineering concerns. M2 now turns those requirements into a controlled solution direction without changing the M1 baseline unless new evidence justifies a formal change (CivicConnect Team, 2026a).

This contribution also aligns with Thaban's completed M2 technology and deployment work. His decision selects a React/TypeScript/Vite browser client, a Java 25 LTS and Spring Boot 4.1.1 backend, PostgreSQL 18.6 persistence, and a container-ready deployment direction while the final hosting provider remains deferred. The architecture below uses those technologies but keeps architecture and technology as separate ideas: the architecture defines responsibilities and boundaries; the technology ADR defines the tools used to implement those boundaries (Ndlozi, 2026).

1.1 Baseline review result

M1 / M2 evidence	Architecture consequence
NFR-002 - authorisation/privacy	The architecture needs one clear security boundary for backend operations and must prevent modules from bypassing role checks.
NFR-004 - auditability	Status, ownership and resolution changes need reliable audit/history recording. The architecture must keep the correctness path explicit.
FR-007 - controlled status transitions	Request lifecycle rules belong in one request-management responsibility rather than being repeated in controllers or UI code.
FR-009 / FR-010 - management oversight and lifecycle information	The architecture needs controlled query/reporting responsibilities without exposing database tables directly to the UI.
C-001 / C-002 - three-person team and fixed milestones	The solution should avoid unnecessary distributed services and operational overhead.
C-006 - lifecycle-wide security	Security must affect boundaries, module access and later tests, not only deployment configuration.
FEC-002 - testability	Module boundaries and interfaces should allow focused unit/integration testing.
FEC-003 / RISK-005 - deployment compatibility	Architecture must be plausible for the container-ready direction selected by Thaban's workstream.
FEC-006 - maintainability/changeability	Business responsibilities should be separated so later changes do not spread through

M1 / M2 evidence	Architecture consequence
	unrelated code.

No M1 scope or requirement change is introduced by this architecture decision. The architecture part of M1 DEC-002 is resolved in M2. The final hosting provider remains deferred under ADR-M2-DEP-001. Detailed database ownership is defined in the data/persistence section and must remain consistent with the architecture boundaries.

2. Architecturally Significant Requirements and Quality Drivers

Not every requirement changes the architecture. The drivers below were selected because they materially affect system boundaries, responsibility allocation, persistence, security, testing or deployment. This avoids making a generic list of quality attributes, as required by the M2 brief (Belgium Campus ITversity, 2026c).

ID	Driver	Source	Measurable / testable expectation	Architecture effect
ASR-01	Authorised access and privacy	NFR-002; C-006	All defined unauthorised-access cases must be denied and authorised roles must see only permitted functions/data.	Keep role checks at controlled backend/application boundaries; no UI or reporting component may bypass authorisation.
ASR-02	Lifecycle auditability and correctness	NFR-004; FR-007; FR-010	For tested ownership/status/resolution changes, actor, action and timestamp must be recorded.	Keep request lifecycle and mandatory audit/history work in one correctness boundary and one relational persistence model.
QD-01	Maintainability and changeability	FEC-006; C-001; C-002	Changes to one business capability should be local where practical and should not require unrelated modules to change.	Use clear business modules and explicit interfaces inside one deployable backend.
QD-02	Testability	C-005; FEC-002	Core lifecycle rules, module interfaces and selected design decisions must be testable independently and through initial integration tests.	Keep domain/application logic out of UI/controllers and use explicit module responsibilities.
QD-03	Deployment simplicity and compatibility	C-003; C-008; RISK-005; FEC-003	The application must remain feasible for the container-ready local/integration direction without a production-provider commitment.	Prefer one backend deployment unit and one relational database over distributed services that add networking and operations complexity.

2.1 Most influential architecture drivers

The strongest drivers are ASR-01 and ASR-02. Authorisation/privacy affects every entry point to protected data and functions, while auditability affects the core request lifecycle and persistence boundary. QD-01 and QD-03 then limit how much architectural complexity is reasonable for a three-person team. Together they favour strong internal boundaries without the cost of independently deployed services.

3. Architecture Alternatives

Three realistic alternatives were considered. The comparison is project-specific. Scores use 1 = weak fit, 3 = acceptable fit and 5 = strong fit. The weighted values are engineering judgement, not objective facts.

Alternative	Description	Main benefit	Main cost / risk
A - Traditional layered monolith	One Spring Boot application organised mainly into controller/service/repository technical layers.	Simple to start and deploy.	Business capabilities can become coupled across shared services; change often crosses several technical layers.
B - Modular monolith	One Spring Boot application, but organised into clear business modules with controlled interfaces and one PostgreSQL database.	Keeps deployment simple while improving cohesion, ownership, testability and change isolation.	Needs discipline to stop modules from reaching into each other's internals; not independent deployment.
C - Microservices	Separate independently deployed services for request, reporting, identity or other capabilities.	Strong deployment independence and service-level isolation when real scale/ownership needs exist.	Network failures, distributed transactions, service security, observability and deployment overhead are not justified by current CivicConnect evidence.

Criterion	Weight	Layered score	Weighted	Modular score	Weighted	Microservices score	Weighted
Authorisation / security boundary	20%	4	16	5	20	4	16
Audit / transaction consistency	20%	5	20	5	20	2	8

Criterion	Weight	Layered score	Weighted	Modular score	Weighted	Microservices score	Weighted
Maintainability / change isolation	20%	3	12	5	20	4	16
Team and schedule fit	15%	5	15	4	12	1	3
Deployment simplicity / cost	10%	5	10	4	8	1	2
Testability	10%	3	6	5	10	3	6
Future integration flexibility	5%	3	3	4	4	5	5
TOTAL	100%		82		94		56

3.1 Selected architecture

Decision: select a modular monolith for the CivicConnect backend, with a separate React browser client and a single PostgreSQL data store. The backend remains one Spring Boot deployment unit, but its internal responsibilities are separated into business-focused modules with controlled interfaces.

This choice is proportional to the current evidence. CivicConnect needs clear security, strong relational transactions, auditability and maintainability, but there is no requirement for independent service deployment or separate team ownership. The Assignment 2 integration research also recommended an in-process interface while the system remains a small single deployment, because adding a network boundary would create cost without proven value (CivicConnect Team, 2026b; Fowler, 2015).

This architecture does not mean that everything is one unstructured codebase. The rule is: one deployable backend, clear internal module ownership, no direct cross-module database access, and explicit interfaces for module collaboration. If later evidence shows a real need for independent deployment, scaling or external integration, a module boundary can be reconsidered through an ADR and controlled change.

4. Architecture Diagrams and Responsibilities

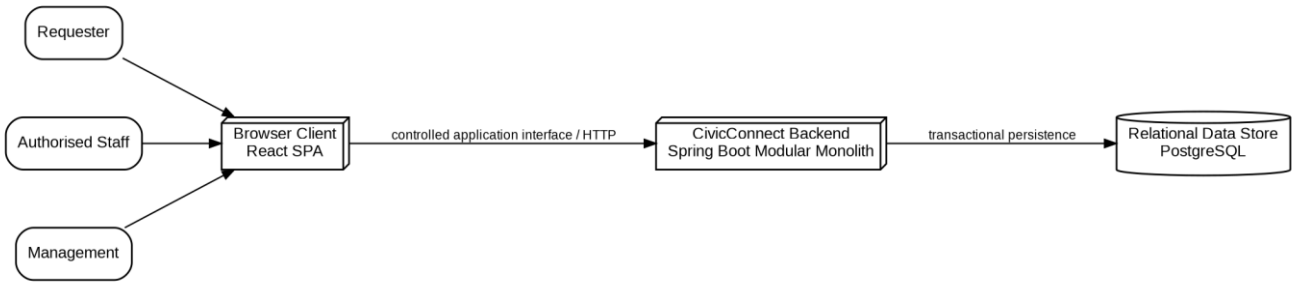


Figure 1. CivicConnect M2 logical container view. Technology labels show alignment with ADR-M2-TECH-001; they do not replace the architecture decision.

Figure 1 separates three main runtime responsibilities: the browser client, the backend application and the relational data store. The final physical hosting layout is not fixed here. That stays consistent with ADR-M2-DEP-001, which keeps the production provider deferred while using a container-ready direction (Ndlozi, 2026).

4.1 Backend module responsibilities

Module / responsibility	Main responsibility	Trace
Access Control	Applies role/authorisation rules at backend application boundaries. Authentication mechanism detail remains a later security/interface decision.	NFR-002; C-006
Request Management	Owns request creation, categorisation, assignment, controlled status transitions, resolution/closure and the correctness path.	FR-001, FR-002, FR-006, FR-007, FR-008
Audit History responsibility	Records mandatory actor/action/timestamp evidence for lifecycle changes in the same correctness transaction.	NFR-004; FR-010
Query & Reporting	Provides authorised requester history/status, staff search/filter/sort and management oversight using controlled read/query services.	FR-003, FR-004, FR-005, FR-009
In-process Application Events	Allows secondary reactions to request changes without making Request Management depend directly on every secondary consumer.	A2 Task 1; design decision ADR-M2-DES-001

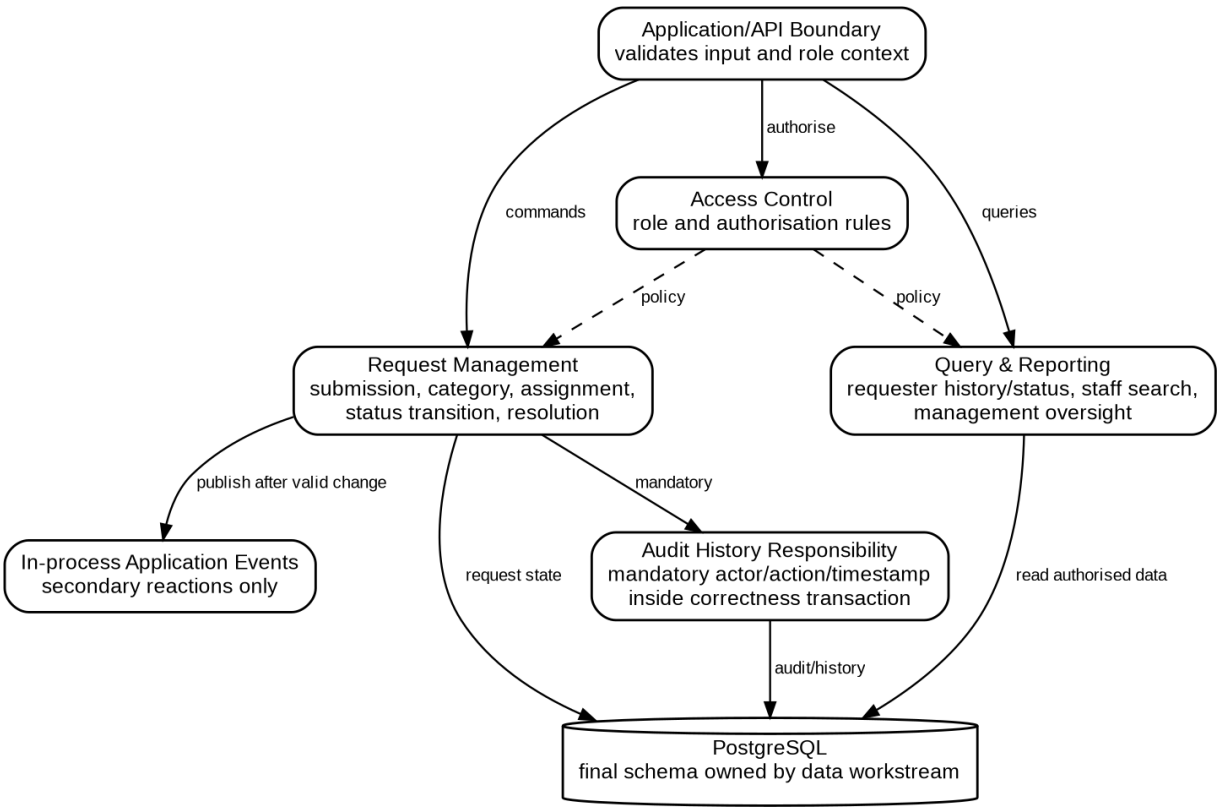


Figure 2. Spring Boot modular-monolith responsibilities. Detailed entities and schema are defined in the data/persistence section.

4.2 Data boundary note

The architecture assumes one PostgreSQL database because strong request/audit transactions are important and the team has not justified distributed data ownership. This does not decide the final schema. The data/persistence section defines the entities, relationships, constraints, ownership and concurrency details, and these remain aligned with the architecture module boundaries.

5. ADR-M2-ARCH-001 - Select a Modular Monolith Architecture

ADR field	Record
Status	M2 baseline architecture decision.
Context	M1 DEC-002 deferred final architecture to M2. M2 now needs enough controlled direction for meaningful development.
Decision drivers	NFR-002 authorisation/privacy; NFR-004 auditability; FR-007 controlled lifecycle; C-001/C-002 team and schedule; C-006 security; C-008 compatibility; FEC-002/FEC-003/FEC-006; RISK-005.
Alternatives	Traditional layered monolith; modular monolith; microservices.
Decision	Use a React browser client with one Spring Boot modular-monolith backend and one PostgreSQL relational data store. Organise backend code by business responsibility with explicit internal interfaces.
Rationale	The choice gives stronger internal boundaries and testability than an unstructured layered monolith, while avoiding the distributed transactions, networking, security and operational overhead of microservices.
Technology alignment	Implements the architecture using Thaban's ADR-M2-TECH-001: React/TypeScript/Vite, Java 25 LTS, Spring Boot 4.1.1 and PostgreSQL 18.6.
Deployment alignment	One backend deployable is compatible with the container-ready direction in ADR-M2-DEP-001. The production hosting provider remains deferred.
Trade-offs	Module boundaries require discipline and review. The backend cannot be deployed module-by-module. A future split would require an ADR and migration work.
Risks	Module boundary erosion; shared-database shortcuts; overgrown shared/common package; later evidence may justify a different boundary.
A2 evidence	Assignment 2 Task 3 compared in-process interfaces, REST and asynchronous messaging and recommended a modular-monolith/in-process interaction until a real distribution need exists.
Resolves / updates	Resolves the architecture part of M1 DEC-002. Technology is resolved by ADR-M2-TECH-001. Hosting-provider selection remains deferred by ADR-M2-DEP-001.

ADR field	Record
Downstream consequence	Repository/package structure, RTM architecture fields, data ownership, design decisions and initial implementation must follow these boundaries.

6. A2-Informed Design Decision 1 - Lifecycle Change Reactions

6.1 Actual design problem

When an authorised staff member changes a request, more than one thing may need to react. The request state changes, audit/history must be recorded, requester-facing feedback may change, management information may need refreshing, and future notifications may react. If Request Management directly calls every current and future reaction, it becomes tightly coupled and gains too many responsibilities.

The important distinction is correctness. Audit/history is mandatory evidence and must stay with the main transaction. Requester feedback refresh, reporting projections and future notifications are secondary reactions and should not make the core lifecycle service depend on every consumer.

6.2 A2 alternatives considered

Alternative	How it works	Benefit	Cost / risk
Direct orchestration	RequestApplicationService changes the request and directly calls every secondary service.	Simple control flow and easy to follow for very few reactions.	Each new reaction adds another dependency and reason for the lifecycle service to change.
Observer / in-process application event	RequestApplicationService publishes a RequestStatusChanged event. Interested handlers react without the request service knowing each handler.	Lower coupling, easier extension and focused tests.	Control flow is less direct; handler failure and ordering rules must be clear. Using events for mandatory correctness work would be unsafe.

6.3 Final M2 decision

Decision: use an in-process Observer/application-event approach for secondary reactions to a successful request lifecycle change. Keep the request update and mandatory audit/history write synchronous and inside the same correctness transaction. Publish a RequestStatusChanged event only for secondary work.

This matches the Assignment 2 recommendation, but M2 accepts it because it fits the selected modular monolith and the current CivicConnect requirements. It reduces coupling without introducing a message broker, another deployable service or eventual consistency on the correctness path. External notifications are still deferred, so no notification provider implementation is claimed.

Under the selected Spring technology, the first implementation can use an in-process Spring application event. The event must carry only the information needed by approved handlers. Asynchronous execution is not required in M2 and should not be added until failure/retry behaviour is justified.

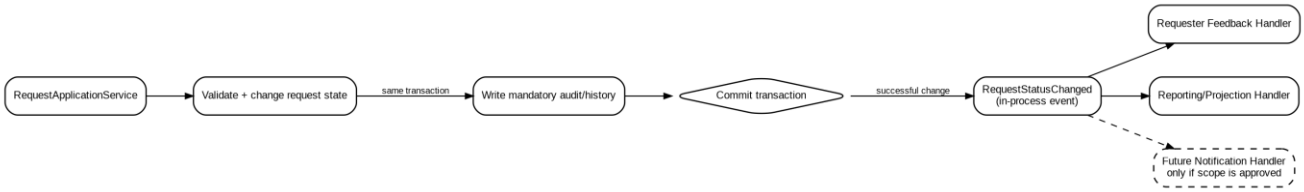


Figure 3. Design Decision 1: mandatory state and audit work stay in the transaction; secondary reactions use an in-process event.

6.4 Affected classes/interfaces - proposed

Proposed element	Responsibility
RequestApplicationService	Coordinates the authorised lifecycle use case and transaction boundary.
Request / lifecycle policy	Checks whether the requested transition is allowed once the final transition catalogue is approved.
AuditHistoryService / audit persistence responsibility	Records mandatory lifecycle evidence in the same transaction.
RequestStatusChanged	Small event describing a successful lifecycle change for secondary handlers.
RequesterFeedbackHandler	Updates or prepares requester-facing feedback where needed.
ManagementProjectionHandler	Updates any later read/projection responsibility if the data design uses one.
Future NotificationHandler	Not implemented unless notifications are brought into scope through controlled change.

7. ADR-M2-DES-001 - Use In-Process Events for Secondary Lifecycle Reactions

ADR field	Record
Status	M2 baseline design decision. Implementation evidence is recorded in the RTM when available.
Design problem	Request lifecycle changes may trigger several secondary reactions, creating coupling if the request service directly depends on every consumer.
A2 evidence	Assignment 2 Task 1 compared direct orchestration with Observer/in-process domain events and recommended events for secondary reactions while keeping audit/history synchronous.
Alternatives	Direct service-to-service orchestration; Observer/in-process application event.
Decision	Use RequestStatusChanged as an in-process event for secondary reactions. Keep state change plus mandatory audit/history in the main transaction.
Why it fits CivicConnect	Supports maintainability/testability quality drivers, fits the modular monolith, avoids a network/message-broker boundary, and preserves transactional auditability.
Expected benefit	Lower coupling; better cohesion in Request Management; easier addition/removal of secondary handlers; focused unit tests.
Trade-offs	Event-driven flow is less visible than direct calls. Handler ordering/failure policy must be documented. Events must not be used to hide mandatory correctness work.
Technology consequence	Can be implemented with Spring's in-process application-event support. No external broker is selected.
Scope consequence	Does not bring deferred external notifications into scope. A future notification handler needs controlled scope/change approval.
Traceability	NFR-004; FR-007; FR-010; FEC-006; A2 Task 1; ADR-M2-ARCH-001.
Implementation status	Planned until the corresponding code/branch/PR exists. Do not mark Implemented in the RTM before that evidence is merged.

8. RTM and Engineering Register Alignment

The table below links the current architecture and design decisions to the RTM fields. Implementation and verification evidence remains controlled by the live repository and RTM, with Planned used where evidence does not yet exist.

Requirement	ASR / driver	Architecture responsibility	Design / ADR	Technology link	Current implementation / verification status
NFR-002	ASR-01	Access Control + controlled backend boundary	ADR-M2-ARCH-001	Spring Security / ADR-M2-TECH-001	Planned until security configuration/tests exist
NFR-004	ASR-02	Request Management + Audit History responsibility	ADR-M2-ARCH-001; ADR-M2-DES-001	Spring Boot + PostgreSQL	Planned until transaction/audit implementation exists
FR-007	ASR-02	Request Management	ADR-M2-DES-001	Spring application/service layer	Planned until transition service/event evidence exists
FR-009	QD-01 / ASR-01	Query & Reporting	ADR-M2-ARCH-001	Spring Boot + PostgreSQL	Planned until management query/UI evidence exists
FR-010	ASR-02	Request Management + Audit History	ADR-M2-ARCH-001; ADR-M2-DES-001	Spring Boot + PostgreSQL	Planned until lifecycle history evidence exists

8.1 Architecture Baseline Summary

Architecture baseline. M2 resolves the architecture part of M1 DEC-002 by selecting a modular monolith for the CivicConnect backend. The solution uses a separate browser client, one Spring Boot backend organised into clear business modules, and one PostgreSQL relational data store. The architecture was selected after comparing a traditional layered monolith, a modular monolith and microservices against authorisation/privacy, auditability, maintainability, testability, team/schedule capacity and deployment simplicity. A modular monolith gives stronger responsibility boundaries than an unstructured monolith without introducing the distributed-system cost that CivicConnect has not justified. This architecture aligns with ADR-M2-TECH-001 and the container-ready direction in ADR-M2-DEP-001; final hosting remains deferred.

Initial design decision 1. Request lifecycle changes use an in-process application event only for secondary reactions. The request state change and mandatory audit/history record remain synchronous and transactional. This decision is informed by Assignment 2 Task 1 and is recorded in ADR-M2-DES-001.

8.2 Architecture Risk and Forward-Engineering Alignment

Item	M2 register alignment
Architecture risk	Module-boundary erosion remains a risk if code bypasses interfaces or grows a large shared/common package. Mitigation uses package-by-business-module structure, architecture review in PRs, and tests/visibility rules where practical.
FEC-006 Maintainability	The modular-monolith direction and initial module boundaries are defined in M2; actual coupling and hotspots remain to be proven by implementation evidence.
DEC-002	The architecture part is resolved by ADR-M2-ARCH-001. Technology is resolved by ADR-M2-TECH-001; the hosting provider remains deferred by ADR-M2-DEP-001.

9. Architecture and Design Summary

This contribution resolves the architecture part of the M1 deferment without changing the approved M1 scope. The recommended CivicConnect architecture is a modular monolith: a React browser client, one Spring Boot backend with clear business modules, and one PostgreSQL database. The choice is driven mainly by authorisation/privacy, lifecycle auditability, maintainability, testability, team capacity and deployment simplicity.

Design Decision 1 uses an in-process Observer/application-event approach for secondary reactions to request lifecycle changes while keeping mandatory request and audit/history updates inside the correctness transaction. The decision follows the Assignment 2 research but is accepted because it fits the final M2 architecture and current CivicConnect constraints.

B2 - Lebogang Masia: Data, Persistence, Design Decision 2, Integration and RTM

1. Data and Persistence Design

The persistence architecture relies on a single relational PostgreSQL 18.6 database to ensure transactional integrity across the service request lifecycle. Schema changes are governed by versioned Flyway migrations rather than uncontrolled schema auto-generation.

1.1 Core Entities and Relationships

The data model separates the current state of a request from its immutable audit history to satisfy strict auditability requirements (NFR-004, FR-010).

- **ServiceRequest:** The central entity tracking the current state, location details, and active ownership of a request.
- **RequestCategory:** A controlled catalogue entity that restricts category selection during submission (FR-002).
- **RequestAuditRecord:** An append-only historical log entity. Every state transition or assignment creates a new audit record in the same database transaction as the ServiceRequest update, recording the actor, action, and timestamp.
- **User / Role Context:** Minimal identity representation linked to requests for ownership (FR-006) and auditing. Final authentication mechanisms remain deferred.

1.2 Schema and Integrity Constraints

- **Foreign Key Constraints:** ServiceRequest enforces a mandatory relationship to a valid RequestCategory (FR-002) and an optional relationship to an assigned staff User.
- **ACID Transactions:** Following Design Decision 1 (ADR-M2-DES-001), any update to a ServiceRequest and its corresponding RequestAuditRecord insertion are bound within a single Spring @Transactional boundary. If the audit write fails, the state change rolls back.

2. A2-Informed Design Decision 2 - Concurrency Control

When multiple authorised staff members access the same service request, concurrent modifications could lead to lost updates or invalid state transitions (FR-007).

2.1 ADR-M2-DES-002 - Optimistic Concurrency for Request Transitions

ADR Field	Record
Status	M2 baseline design decision.
Design problem	Multiple staff members may attempt to claim, update, or close the same request simultaneously, potentially overwriting each other's changes and corrupting the lifecycle state (FR-007).
A2 evidence	Assignment 2 task research recommends atomic status + audit persistence coupled with optimistic concurrency control for relational web applications.
Alternatives	Pessimistic Locking (database row locks); Optimistic Concurrency (version column).
Decision	Implement Optimistic Concurrency using a @Version field on the ServiceRequest entity.
Why it fits CivicConnect	Provides safe concurrent access without the performance bottlenecks or deadlocks associated with pessimistic locking. Fits naturally into Spring Data JPA and PostgreSQL.
Expected benefit	Prevents lost updates. If Staff A and Staff B open a request, and Staff A resolves it, Staff B's subsequent

	attempt to reassign it will fail with an OptimisticLockException, prompting them to refresh the latest state.
Trade-offs	The React UI must handle 409 Conflict HTTP responses gracefully and prompt the user to refresh the data.
Traceability	FR-007; NFR-004; FEC-004; ADR-M2-TECH-001.

3. Integration and API Contracts

The Spring Boot backend and React SPA interact via a controlled RESTful HTTP application interface. The endpoints encapsulate the Request Management module boundaries.

3.1 Primary API Endpoints

- **Submit Request (POST /api/requests):**
  - **Purpose:** Accepts initial request payload from requesters (FR-001).
  - **Payload:** Includes categoryId, description, locationData.
  - **Response:** 201 Created with a unique request identifier.
- **Query Requests (GET /api/requests):**
  - **Purpose:** Provides search, filter, and sort capabilities for staff (FR-005) and management oversight (FR-009).
  - **Integration:** Relies on Spring Data JPA specifications to translate query parameters (e.g., ?status=OPEN&category=IT) into secure database queries without exposing internal schema.
- **Update Request Status (PATCH /api/requests/{id}/status):**
  - **Purpose:** Executes controlled status transitions (FR-007).
  - **Payload:** Includes targetStatus, actionComment, and the version attribute to satisfy ADR-M2-DES-002.
  - **Response:** 200 OK on success, or 409 Conflict if the provided version does not match the database version.

4. Updated Requirements Traceability Matrix (RTM)

The RTM is evolved from the Milestone 1 baseline to include Milestone 2 architecture, design, and technology decisions. Implementation and Verification fields remain deferred to M3/M4.

Source	Req ID	Requirement	Design / Architecture (M2)	Technology (M2)	Implementation (M3)
STK-01	FR-001	Submit service request	ADR-M2-ARCH-001 (Request Mgmt Module)	Spring Boot REST, PostgreSQL	Deferred
STK-01	FR-002	Controlled categorisation	Relational FK to RequestCategory	Spring Data JPA	Deferred
STK-02	FR-005	Staff view/search/filter	ADR-M2-ARCH-001 (Query & Reporting)	React SPA, Spring Web	Deferred
STK-02	FR-007	Controlled status/actions	ADR-M2-DES-001, ADR-M2-DES-002	Spring @Transactional, @Version	Deferred
STK-03	FR-009	Management oversight	ADR-M2-ARCH-001 (Query & Reporting)	React SPA, PostgreSQL	Deferred
STK-03	FR-010	Lifecycle accountability	ADR-M2-DES-001 (Sync Audit)	Spring Data JPA	Deferred
C-006	NFR-002	Role-based access/privacy	ADR-M2-ARCH-001 (Access Control)	Spring Security	Deferred
C-005	NFR-004	Auditability of	RequestAuditRecord Entity, ADR-M2-	PostgreSQL 18.6	Deferred

		changes	DES-001		

B3 - Thaban Ndlozi: Technology, Deployment, SCM/CI and Engineering Control

1. Purpose, Evidence Basis and Status

This section defines CivicConnect's Milestone 2 technology selection, deployment compatibility, configuration management and engineering controls. It forms part of PED v2.0 and the controlled repository evidence. Milestone 2 requires an evidence-based technology-stack decision, current versions and dependencies, deployment compatibility reasoning, updated risks/forward considerations, continued protected-main governance, and meaningful initial automated verification (Belgium Campus ITversity, 2026c).

The work deliberately preserves the M1 decision history. In M1, DEC-002 deferred final architecture, technology stack and delivery platform until M2 because the team did not yet have sufficient project-specific evidence. RISK-005 and FEC-003 also recorded the possibility of technology/environment incompatibility. This M2 contribution resolves the technology-stack decision using the approved M1 requirements and constraints, current official technology evidence, and the Assignment 2 research.

Repository evidence is reported only where it exists in the controlled repository. A CI workflow, application bootstrap, database migration, health check or deployment is treated as implemented only when it can be shown in reviewed project evidence such as a Pull Request or workflow run.

1.1 Source and project drivers used

M1 evidence	Technology / deployment consequence
NFR-002 - authorised access/privacy	Prefer technology with strong, maintainable authentication/authorisation support and clear security controls.
NFR-003 - secrets/configuration security	Externalise credentials and configuration; no secrets in source control or CI files.
NFR-004 - auditability of lifecycle changes	Persistence and backend stack must support transactional application logic and reliable audit-history storage.
C-001 / C-002 - three-person team and fixed milestones	Avoid unnecessary distributed infrastructure and favour tooling that supports fast, repeatable development and review.
C-003 - low-cost resources	Prefer no-license-fee core technologies and defer paid hosting commitments until cost evidence exists.
C-006 - lifecycle-wide security	Security affects framework selection, CI, dependency management, configuration and later deployment.
C-008 - institutional environment compatibility	Prove local/container compatibility before treating the stack as operationally safe.
C-009 - protected main and two reviews	CI must support, not replace, the existing Pull Request and two-non-author approval controls.
RISK-005 / FEC-003 / DEC-002	M2 must resolve the deferred stack decision while keeping residual hosting/environment risks visible.

1.2 M2 control boundary

Architecture and data artefacts remain shared team decisions and are kept consistent with the technology baseline in PED v2.0.

Production hosting provider selection is deliberately deferred. M2 requires a credible deployment direction, not a production deployment.

Authentication implementation detail (for example session-based versus token-based browser authentication) remains an interface/security design decision; Spring Security is selected as the security framework without inventing that later decision.

Exact resolved transitive dependency versions must be preserved by the build files/lockfiles and reviewable in the repository.

2. Technology-Stack Decision

2.1 Alternatives considered

The Assignment 1 research compared a Java/Spring Boot stack with a TypeScript/NestJS stack and concluded that both were viable under a neutral context. M2 now has project-specific evidence: CivicConnect has high-priority authorisation/privacy and auditability requirements, a relational request lifecycle, a three-person team, low-cost constraints, and a need for controlled deployment and CI. The comparison below therefore re-evaluates the backend alternatives for CivicConnect rather than copying the A1 result.

Alternative	Proposed composition
Alternative A	React SPA + TypeScript frontend; Java 25 LTS; Spring Boot 4.1.1 backend; Spring Security; Spring Data JPA; PostgreSQL 18.6.
Alternative B	React SPA + TypeScript frontend; Node.js 24 LTS; NestJS 12.1.x backend; Nest security middleware/guards; PostgreSQL 18.6.

React and PostgreSQL are kept common to both alternatives so that the comparison focuses on the main backend/runtime decision. Current official documentation identifies Spring Boot 4.1.1 as stable and compatible with Java 17 through 26; Java 25 is an LTS release. NestJS 12 is also current, but version 12 was released recently and introduces ESM-centred package and tooling changes. These facts do not make one framework universally superior; they affect upgrade/dependency risk in this project's schedule (Spring, 2026a; Oracle, 2026; NestJS, 2026).

2.2 CivicConnect decision criteria and weighted comparison

Scoring: 1 = weak fit, 3 = acceptable/neutral fit, 5 = strong fit. Scores are engineering judgements supported by evidence, not objective facts. Team capability is scored neutrally because no verified evidence in the controlled project artefacts currently demonstrates a decisive Java-versus-TypeScript skill advantage.

Criterion	Weight	Spring score	Spring weighted	NestJS score	NestJS weighted	Project-specific rationale

Criterion	Weight	Spring score	Spring weighted	NestJS score	NestJS weighted	Project-specific rationale
Security / authorisation	20%	5	20.0	4	16.0	Spring Security provides first-class authentication, authorisation and exploit-protection capabilities; both stacks are viable but Nest relies more heavily on composition/configuration of packages.
Data integrity / transaction fit	15%	5	15.0	5	15.0	Both can use PostgreSQL transactions/constraints; Spring's transaction/JPA integration is highly mature, but the capability exists in both.
Maintainability / modularity	15%	5	15.0	5	15.0	Both provide dependency injection and modular structuring. Architecture discipline matters more than framework branding.
Testing / CI tooling	10%	5	10.0	5	10.0	Both have mature unit/integration testing and can run effectively in GitHub Actions.
Deployment compatibility	10%	5	10.0	5	10.0	Both can run as containers and on common Linux/Windows development environments.
Verified team capability	15%	3	9.0	3	9.0	No controlled evidence currently justifies favouring one stack on skill alone; bootstrap evidence must confirm the decision remains feasible.
Development / licence cost	10%	4	8.0	5	10.0	Both avoid a paid database licence; Nest may reduce language switching. Hosting cost remains workload/provider dependent.
Dependency / upgrade risk	5%	4	4.0	3	3.0	Both require maintenance; NestJS 12 is a newly released major with ESM/toolchain migration implications, increasing short-term schedule risk.
TOTAL	100%		91.0		88.0	

2.3 Selected stack

Decision: select the Java/Spring Boot alternative for the M2 CivicConnect technology baseline. The margin is deliberately treated as modest rather than absolute. The decision is driven mainly by CivicConnect's high-priority role/authorisation requirement, lifecycle auditability, relational persistence, and the benefit of Spring's integrated security, transaction and data-access conventions. NestJS remains a viable alternative and should not be described as unsuitable. If bootstrap evidence demonstrates a material team-capability disadvantage for Spring before baseline sign-off, the team should reopen the ADR rather than conceal the new evidence.

Layer / concern	Selected technology	M2 rationale / control
Client UI	React 19.3.0	Component-based browser UI. React 19.3 is the current major/minor documented by the React team as of September 2026.
Client language	TypeScript 5.9	Static typing for UI code and API contracts where appropriate.
Client build	Vite 8.1.x	Current Vite 8 generation; fast development server and production build pipeline.
Frontend tooling runtime	Node.js 24 LTS	LTS runtime for Vite/npm tooling; production CivicConnect backend is not Node-based under this decision.
Backend language/runtime	Java 25 LTS (OpenJDK distribution)	Current LTS line; supported by Spring Boot 4.1.1. Use an OpenJDK distribution to avoid unnecessary vendor lock-in.
Backend framework	Spring Boot 4.1.1	Stable release; stand-alone application support, externalised configuration, health/metrics features and strong Spring ecosystem integration.
Security	Spring Security (Boot-managed compatible version)	Authentication/authorisation framework. Do not override the version independently unless a documented compatibility/security reason exists.
Persistence	Spring Data JPA 4.1.x + Hibernate version managed by Spring Boot	Repository/data-access abstraction while retaining explicit transaction and domain rules in application services.
Database	PostgreSQL 18.6	Current supported PostgreSQL 18 patch release; relational constraints and transactions fit request/audit data integrity.
Schema migration	Flyway (version managed by Spring Boot)	Versioned database migrations rather than relying on uncontrolled schema auto-generation.
Backend build	Maven Wrapper; Maven 3.9.16 toolchain	Repeatable build entry point; wrapper should be committed to the repository.
Backend tests	JUnit 5 / Spring Boot Test	Unit/integration verification using dependencies managed by Spring Boot.
Frontend tests	Vitest 5.x	Vite-native test runner; current supported generation.
CI	GitHub Actions	PR-visible automated build/test checks integrated with the existing protected-main workflow.
Container/dev orchestration	Docker + Docker Compose v2	Reproducible multi-service local/test environment for backend + PostgreSQL and later application services where needed.

3. ADR-M2-TECH-001 - Select the CivicConnect Technology Stack

ADR field	Record
Status	M2 baseline technology decision.
Context	M1 deliberately deferred the final technology stack. M2 now requires a frontend, backend/runtime, persistence and supporting build/testing/deployment-compatible technology decision.
Decision drivers	NFR-002 authorised access/privacy; NFR-003 configuration security; NFR-004 auditability; C-001 three-person team; C-002 schedule; C-003 low cost; C-006 security; C-008 environment compatibility; RISK-005; FEC-003.
Alternatives	A: React + Java/Spring Boot + PostgreSQL. B: React + TypeScript/NestJS + PostgreSQL.
Decision	Adopt React 19.3 + TypeScript 5.9 + Vite 8.1 for the browser UI; Java 25 LTS + Spring Boot 4.1.1 for the backend; PostgreSQL 18.6 for persistence; Maven Wrapper, Spring Boot Test/JUnit, Vitest and GitHub Actions for build/test/CI support.
Rationale	The selected stack has the stronger project-specific fit for integrated security, transactional/audit-oriented backend logic and maintainable relational persistence while remaining container/deployment compatible. The earlier A1 evidence showed both alternatives viable; M2 project constraints make the Spring option the stronger current judgement.
Trade-offs	Two primary implementation languages increase context switching. The Spring ecosystem is convention-rich and may create a learning burden if team capability is weaker than assumed. Java/Spring container memory may be higher than a minimal Node service. These costs are accepted because the system is a three-person modular application, not a large distributed platform.
Risks	Team learning curve; dependency/patch drift; hosting compatibility; configuration/secrets errors; later evidence showing stronger TypeScript capability.
Controls	Small bootstrap spike; Maven wrapper; locked frontend dependencies; dependency review; current supported patch versions; containerised local environment; CI on PRs; no secrets in repository.
Supersedes / resolves	Resolves the M1 deferment recorded by DEC-002 for technology stack selection. The hosting-provider part of DEC-002 remains intentionally deferred under ADR-M2-DEP-001.
Downstream consequences	Architecture/data/design artefacts must align with the selected runtime and database; RTM technology fields can now reference ADR-M2-TECH-001; application bootstrap and CI can begin; M3 must provide stronger automated/security/deployment evidence.

4. Deployment Compatibility Baseline

M2 requires a credible deployment direction but does not require production deployment. The selected stack is therefore baselined as container-ready, while the final hosting provider is intentionally deferred. Spring Boot can run as a stand-alone application with an embedded servlet container, and Docker Compose can define the application services, network and persistent volumes in one configuration. This provides a consistent development/test path without prematurely purchasing or committing to a cloud platform (Spring, 2026a; Docker, 2026).

Environment	M2 direction	Configuration / state rule	Evidence expected
Developer workstation	React/Vite dev server + Spring Boot + PostgreSQL (prefer PostgreSQL via Docker Compose).	Local-only values may use `.env` or local profile files that are excluded from Git; commit only safe examples/defaults.	README setup steps; compose file; successful local startup; no secrets committed.
Automated CI	GitHub-hosted runner builds backend and frontend; persistence tests use a disposable PostgreSQL service/container where needed.	CI secrets injected through GitHub secrets/environment controls; tests use non-production credentials.	PR check results; reproducible build commands; test output.
Staging direction	Container-capable host for backend plus PostgreSQL; frontend may be static-hosted or served through a reverse proxy depending on final architecture.	Environment-specific variables; health endpoint; database migration step; rollback/recovery plan developed progressively.	M2 compatibility rationale; M3 staging deployment evidence.
Production	Not selected in M2.	Provider, region, backup service, TLS termination, monitoring and cost require later evidence.	Deliberate deferment recorded in ADR/risk/forward-engineering records.

4.1 Configuration and secrets policy

No password, API key, token, private key or production connection string is committed to Git.
Spring Boot configuration is externalised. Repository files contain safe defaults or variable names, not live credentials.
Frontend build-time environment variables must never contain server-side secrets; anything exposed to browser code is treated as public.
GitHub Actions secrets/environments are used only when a workflow genuinely requires a secret; workflow permissions should use least privilege.
Provide `.env.example` or equivalent documentation with variable names and safe example values.
Database schema changes are applied through versioned migrations; production-like environments must not depend on `ddl-auto=create` or ad hoc manual changes.

4.2 ADR-M2-DEP-001 - Container-ready direction; provider deferred

ADR field	Record
Context	M1 deferred the delivery platform because hosting/environment evidence was incomplete. M2 needs compatibility evidence without over-implementing deployment.
Decision	Adopt a container-ready deployment direction using Docker/Compose for repeatable local and integration environments. Keep the final staging/production provider unselected until cost, environment availability, networking, persistence, backup and operational evidence are confirmed.
Why proportionate	The three-person team needs reproducibility more than orchestration complexity. Docker/Compose supports multiple services without introducing Kubernetes or microservice infrastructure that CivicConnect has not justified.
Trade-off	Containerisation adds image/build configuration and local Docker resource use. Provider deferral means final network/storage/backup details remain open.
Evidence still required	BC/environment compatibility; chosen hosting provider limits/cost; managed PostgreSQL availability or alternative; TLS/domain approach; backup/restore; staging-production parity; monitoring/alerts.
Downstream consequence	M3 should establish staging, migrations, secret handling, rollback/recovery, health checks and environment-parity evidence before release-readiness claims.

5. Configuration Management and Continuous Integration Control

The M1 GitHub governance remains in force: protected main, substantive changes through Pull Requests, two approvals from the non-author members, meaningful review and resolved conversations. M2 adds automated verification to this human control. Assignment 2 research recommended short-lived task branches around protected main and PR-visible CI; M2 explicitly allows progressive CI rather than requiring a mature CI/CD pipeline.

5.1 Recommended integration workflow

Create or confirm a meaningful issue/task with owner, done criteria and requirement/ADR references.
Create a short-lived task branch from the latest `main`.
Commit small coherent changes with meaningful messages and push progressively.
Open a Pull Request early enough for review; keep it focused on one engineering concern.
Run CI automatically on every PR targeting `main`.
Fix blocking failures before final approval; do not merge a red build.
Both non-author team members perform meaningful review; resolve conversations and obtain two approvals.
Merge into protected `main`; CI runs again on `main`, then update RTM/ADR/application evidence where relevant.

5.2 Initial CI quality gates

Check	Initial policy	Reason
Backend compile/build + tests	BLOCK	`./mvnw -B verify` must succeed. A non-building backend is not integrable.
Frontend clean install + build	BLOCK	`npm ci` followed by the agreed production build must succeed using the committed lockfile.
Frontend unit tests	BLOCK once initial stable test suite exists	Implemented behaviour should not regress silently. During first bootstrap PR, configure the command before making it a required branch check.
Database migration smoke check	BLOCK when migrations exist	A clean database must accept the versioned migration sequence.
Lint / static analysis	BLOCK after rule set is agreed	Deterministic rules can protect quality, but noisy/unreviewed rules should not create meaningless failures.
Dependency review	WARN initially; later BLOCK new High/Critical findings	Use PR dependency review to expose vulnerable additions; tighten the policy once signal is stable.
Secret detection	BLOCK credible findings	Credentials must not enter repository history.

Check	Initial policy	Reason
Coverage percentage	WARN initially	M2 does not require mature coverage. Use critical-path tests first; baseline a threshold only when it is meaningful.

5.3 Repository ruleset change for M2

Do not enable a required status check before the workflow exists and has completed successfully at least once, because that can block merging with a check that GitHub cannot satisfy. After the first stable CI workflow run, update the existing `main` ruleset to require the stable build/test check names in addition to the current two-reviewer and conversation-resolution controls.

Keep: protected `main`, two non-author approvals, dismissal of stale approvals, resolved conversations, blocked force pushes and deletion protection.
Add after CI stabilises: required status checks for the backend build/test and frontend build/test jobs.
Add progressively: dependency/security status check when the team has tuned severity and false-positive handling.
Do not replace peer review with CI. Automated checks prove only what they are designed to test.

5.4 ADR-M2-CI-001 - Introduce PR-visible CI

ADR field	Record
Context	Coding and integration are increasing. Manual review alone cannot repeatedly verify every build/test step, while the project still requires two independent human approvals.
Decision	Use GitHub Actions for CI on Pull Requests to `main` and on pushes to `main`. Build and implemented unit tests are blocking; migration/lint/security gates become blocking when the underlying check is stable and meaningful.
Alternatives considered	Manual local verification only; CI on `main` only; PR + main CI.
Rationale	PR CI detects integration failures before merge and makes results visible to reviewers. Running again on main verifies the merged state.
Trade-offs	Workflow maintenance and runner time; risk of noisy/flaky checks; dependency actions themselves require maintenance.
Security controls	Least-privilege workflow permissions; secrets stored in GitHub secrets/environments; dependencies/actions kept current; no secrets printed to logs.
Downstream consequence	Once stable, required status checks become part of the protected-main gate. M3 can strengthen static analysis, dependency/security scanning, coverage and staging evidence.

6. M2 Risk Register Update - Technology and Engineering Control

The M1 Risk Register remains live. The entries below extend the single controlled register with M2 technology, deployment and SCM risks.

ID	Risk	Cause	P	I	Score / Priority	Owner	Mitigation	Contingency	Status
RISK-005 (updated)	Selected stack or deployment direction may still prove incompatible with available development/hosting environments.	Final provider and institutional compatibility are not yet demonstrated.	2	4	8 / Medium	T. Ndlozi	Bootstrap the stack on team machines; use containerised PostgreSQL; verify build on CI; keep provider selection deferred until evidence.	If a blocking incompatibility appears, record an ADR/change, select the nearest viable alternative and update schedule/RTM/risks.	Monitoring
RISK-009	Dependency or security-patch drift may introduce vulnerabilities or incompatible upgrades.	Spring/React/npm/database ecosystems change during the project; transitive dependencies may carry vulnerabilities.	3	4	12 / High	T. Ndlozi	Pin supported major/minor choices; use Maven dependency management and npm lockfile; review Dependabot/dependency-review findings; update through PRs.	Patch/replace affected dependency; re-run verification; document any material framework/version change in ADR/risk records.	Open
RISK-010	CI may slow integration or create false confidence if checks are flaky, noisy or incomplete.	New automation may be misconfigured or treated as a substitute for review.	3	3	9 / Medium	T. Ndlozi	Start with deterministic build/tests; tune checks before making them required; retain meaningful two-person review.	Temporarily remove only the faulty required check through controlled governance change, repair it, and re-enable after evidence.	Open

ID	Risk	Cause	P	I	Score / Priority	Owner	Mitigation	Contingency	Status
RISK-011	Secrets or environment-specific configuration may be exposed or misapplied.	Local `.env` files, workflow logs or copied configuration can leak credentials or point an environment at the wrong service.	2	5	10 / High	T. Ndlozi	Ignore secret files; use safe examples; GitHub secrets/environments; least privilege; review workflow logs and config changes.	Rotate exposed credentials immediately; remove/rewrite compromised history where required; investigate impact and record incident/corrective action.	Open
RISK-012	Hosting or managed database cost/limits may make the selected deployment provider unsuitable.	Final workload, storage, backup and free-tier limits are still uncertain.	3	3	9 / Medium	T. Ndlozi	Keep provider choice deferred; compare limits/cost against measured or justified needs before staging/production commitment.	Move to another container-capable provider or adjust scope/resource plan through controlled decision and change records.	Open
RISK-013	Team learning curve for the selected Spring stack may reduce delivery speed.	No controlled evidence yet proves a decisive team capability advantage for Java/Spring.	3	4	12 / High	T. Ndlozi	Use a small bootstrap spike, shared conventions, pair review and targeted learning before broad implementation; keep modules simple.	If bootstrap evidence shows material schedule risk, reopen ADR-M2-TECH-001 before baseline sign-off and compare the NestJS alternative using actual team evidence.	Open

Scoring convention preserved from M1: Probability 1-5 x Impact 1-5. Priority bands: 1-4 Low, 5-9 Medium, 10-14 High, 15-25 Critical.

7. Forward Engineering Considerations - M2 Update

ID / consideration	M2 status	Evidence now available	Information still missing	Next lifecycle action	Residual risk
FEC-001 Security, Authentication & Role-Based Access	Partially resolved	Spring Security selected; NFR-002 remains the driver.	Exact authentication flow, final role/permission matrix, password/account policy, session/token lifetime, sensitive-data classification.	M2/M3 security design and tests; threat review; access-control verification.	Do not treat framework selection as proof of secure configuration.
FEC-002 Testability & Verification Strategy	Partially resolved	JUnit/Spring Boot Test, Vitest and GitHub Actions selected as the verification toolchain.	Final test environment, performance targets, representative test data, coverage policy, end-to-end strategy.	M2 initial tests; M3 automated verification and quality gates.	Vague requirements or weak tests could create false confidence.
FEC-003 Deployment Environment & Platform Compatibility	Partially resolved	Container-ready direction selected; Java/Spring/PostgreSQL versions identified.	Final host/provider, networking, TLS/domain, managed database choice, storage, backup and cost limits.	M3 staging and environment-parity evidence.	Provider incompatibility or cost can still force change.
FEC-004 Data Architecture, Audit History & Recovery	Partially resolved	PostgreSQL selected; A2 recommends atomic status + audit persistence and optimistic concurrency.	Final schema ownership, retention, recovery point/time objectives, backup schedule, migration/restore evidence.	Data model/migrations in M2; recovery evidence in M3.	A DB choice alone does not prove integrity or recoverability.
FEC-005 Observability, Logging & Operational Support	Open	Spring Boot provides health/metrics support that can be used later.	Monitoring platform, alert thresholds, log retention, support responsibilities, incident process.	M2 basic health/log structure; M3 staging monitoring/readiness.	Failures may remain invisible or difficult to diagnose.
FEC-006 Maintainability, Changeability & Technical Debt	Ongoing	Versioned build, DI/modularity, PR review and dependency controls support maintainability.	Actual coupling/hotspots, change history, dependency churn, technical-debt evidence.	M2 design reviews; M3 debt register/refactoring decisions.	Framework choice cannot compensate for poor module boundaries or uncontrolled change.

8. Engineering Decision Log Updates

Decision ID	Decision topic	Status	M2 record / consequence
DEC-002 (M1)	Technology/architecture/delivery platform deferred to M2.	PARTIALLY RESOLVED	Technology selection resolved by ADR-M2-TECH-001; deployment-provider selection remains deferred under ADR-M2-DEP-001. Architecture is resolved by the team's architecture workstream.
DEC-M2-005	Technology stack selection	DECIDED	React 19.3 + TypeScript 5.9 + Vite 8.1; Java 25 LTS + Spring Boot 4.1.1; PostgreSQL 18.6; supporting build/test/CI stack as listed in Section 2.3.

Decision ID	Decision topic	Status	M2 record / consequence
DEC-M2-006	Deployment direction	DECIDED / PROVIDER DEFERRED	Container-ready local/integration direction using Docker/Compose. Final staging/production provider deferred until compatibility, cost, networking, persistence and operational evidence.
DEC-M2-007	Initial CI control	DECIDED	GitHub Actions on PRs to `main` and on main after merge; build and implemented tests block integration; additional gates become blocking when stable.

9. RTM Technology and Control Alignment

Requirement / evidence	M2 technology / control link	Expected application / verification evidence
NFR-002 Authorisation/privacy	Spring Security; ADR-M2-TECH-001; security/role design ADR when finalised.	Security configuration classes; role/authorisation tests; denied unauthorised-access test evidence.
NFR-003 Secrets/configuration security	Externalised Spring configuration; GitHub secrets/environment controls; ADR-M2-CI-001.	`.env.example`; ignored secret files; workflow config; repository secret scan/review evidence.
NFR-004 Auditability	PostgreSQL + transactional service design; A2 persistence evidence; data ADR.	Migration/schema; lifecycle history entity/table; transaction tests.
FR-007 Controlled status transitions	Spring service/domain implementation; TransitionPolicy decision from A2; ADR/design link.	Transition policy class/service; unit tests; persistence transaction evidence.
FEC-003 / RISK-005	ADR-M2-TECH-001 + ADR-M2-DEP-001; selected runtime/database versions.	Successful clean build; local container startup; later staging compatibility evidence.

9.1 Technology and Deployment Baseline Summary

Technology and deployment baseline. M2 resolves the M1 technology-stack deferment by selecting a React/TypeScript/Vite browser client, Java 25 LTS with Spring Boot 4.1.1 for the backend, and PostgreSQL 18.6 for persistence. The choice was made after comparing the previously researched Spring Boot and NestJS alternatives against CivicConnect security, auditability, maintainability, testing, deployment, cost, dependency and team-capability constraints. Spring was selected because the current project evidence gives greater weight to integrated security and transactional/persistence conventions; team capability remains a monitored risk rather than an invented advantage. The application is container-ready for reproducible local/integration environments, while the final hosting provider remains deliberately deferred until environment, cost, networking, backup and operational evidence is available. GitHub Actions is adopted as the initial CI mechanism, complementing rather than replacing the existing protected-main and two-non-author review controls.

10. Technology and Engineering Control Summary

This M2 work package turns the M1 technology/deployment deferment into a controlled engineering direction. The recommended CivicConnect baseline is a React/TypeScript/Vite client, Java 25 LTS and Spring Boot 4.1.1 backend, PostgreSQL 18.6 persistence, Maven/standard test tooling, GitHub Actions CI and a container-ready development/integration environment. The decision is intentionally proportional: it does not introduce microservices, Kubernetes, a production hosting provider or mature CI/CD without evidence. Residual risks - team learning curve, dependency/security drift, secrets/configuration, provider compatibility and cost - remain explicit and actionable.

The key M2 control is traceability: the stack decision resolves M1 DEC-002, updates RISK-005 and FEC-003, supports NFR-002/NFR-003/NFR-004, enables initial application/CI evidence, and provides concrete technology references for the evolving RTM and PED v2.0. If later evidence contradicts the current choice, the correct response is a controlled ADR/change with preserved history, not silently rewriting the M2 baseline.

PART C - M2 BASELINE REVIEW AND SIGN-OFF

Baseline type: Milestone 2 - Architecture, Technology & Initial Design Baseline. Target controlled version: PED v2.0.

Team Sign-off

Team member	Student number	Signature / approval	Date
Thaban Ndlozi	602341	T.Ndlozi	30/09/2026

Team member	Student number	Signature / approval	Date
Anele Nkayi	577168	A.Nkayi	30/09/2026
Lebogang Masia	601562	L.Masia	30/09/2026

References

Apache Maven Project (2026) Downloading Apache Maven. Available at: https://maven.apache.org/download.cgi (Accessed: 24 September 2026).

Belgium Campus ITversity (2026a) SEN381 CivicConnect Master Project Brief: Integrated Team Software Engineering Project. Unpublished module project brief. Pretoria: Belgium Campus ITversity.

Belgium Campus ITversity (2026b) SEN381 CivicConnect Project Milestone 1: Engineering Foundation & Requirements Baseline. Unpublished milestone brief. Pretoria: Belgium Campus ITversity.

Belgium Campus ITversity (2026c) SEN381 CivicConnect Milestone 2: Architecture, Technology & Initial Design Baseline. Unpublished milestone brief. Pretoria: Belgium Campus ITversity.

CivicConnect Team (2026a) CivicConnect Milestone 1 Project Engineering Document v1.0. Controlled project artefact.

CivicConnect Team (2026b) SEN381 Assignment 2: Research for Engineering Decisions - CivicConnect research brief. Team research artefact.

Docker (2026) Docker Compose. Available at: https://docs.docker.com/compose/ (Accessed: 24 September 2026).

Fowler, M. (2015) 'Monolith First'. MartinFowler.com, 3 June. Available at: https://martinfowler.com/bliki/MonolithFirst.html (Accessed: 25 September 2026).

GitHub (2026a) Continuous integration. Available at: https://docs.github.com/en/actions/get-started/continuous-integration (Accessed: 24 September 2026).

GitHub (2026b) Dependency review. Available at: https://docs.github.com/en/code-security/concepts/supply-chain-security/dependency-review (Accessed: 24 September 2026).

GitHub (2026c) Secure use reference. Available at: https://docs.github.com/en/actions/reference/security/secure-use (Accessed: 24 September 2026).

Microsoft (2026) 'Observer design pattern'. Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/standard/events/observer-design-pattern (Accessed: 25 September 2026).

Ndlozi, T. (2026) CivicConnect M2: Technology, Deployment, Configuration Management and Engineering Control Work Package. Controlled team contribution, 24 September 2026.

NestJS (2026) Migration guide: NestJS 11 to 12. Available at: https://docs.nestjs.com/migration-guide (Accessed: 24 September 2026).

Node.js (2026) Node.js Releases. Available at: https://nodejs.org/en/about/previous-releases (Accessed: 24 September 2026).

Oracle (2026) Oracle Java SE Support Roadmap. Updated 15 September 2026. Available at: https://www.oracle.com/java/technologies/java-se-support-roadmap.html (Accessed: 24 September 2026).

PostgreSQL Global Development Group (2026a) PostgreSQL 18.6 Release Notes. Available at: https://www.postgresql.org/docs/release/18.6/ (Accessed: 24 September 2026).

PostgreSQL Global Development Group (2026b) PostgreSQL License. Available at: https://www.postgresql.org/about/licence/ (Accessed: 24 September 2026).

React Team (2026) React Versions: latest version 19.3. Available at: https://react.dev/versions (Accessed: 24 September 2026).

Spring (2026a) Spring Boot 4.1.1 - System Requirements. Available at: https://docs.spring.io/spring-boot/system-requirements.html (Accessed: 24 September 2026).

Spring (2026b) Spring Boot. Available at: https://spring.io/projects/spring-boot (Accessed: 24 September 2026).

Spring (2026c) Spring Security Reference. Available at: https://docs.spring.io/spring-security/reference/ (Accessed: 24 September 2026).

Spring Data (2026) Spring Data JPA. Available at: https://docs.spring.io/spring-data/jpa/reference/ (Accessed: 24 September 2026).

TypeScript (2026) TypeScript 5.9 Release Notes. Available at: https://www.typescriptlang.org/docs/handbook/release-notes/typescript-5-9.html (Accessed: 24 September 2026).

Vite (2026a) Vite 8.1 is out. Available at: https://vite.dev/blog/announcing-vite8-1 (Accessed: 24 September 2026).

Vite (2026b) Getting Started. Available at: https://vite.dev/guide/ (Accessed: 24 September 2026).

Vitest (2026) Vitest 5.0. Available at: https://vitest.dev/blog/vitest-5 (Accessed: 24 September 2026).

PART D — MILESTONE 3 CONTINUATION

Evidence-based additions to the original PED v2.0 historical baseline

D0. Document continuity and submission baseline

Version 3.0 continues the approved CivicConnect Project Engineering Document (PED) v2.0. The original 31-page PED v2.0 has been preserved intact in the middle of this expanded PDF, retaining its M1/M2 requirements, engineering decisions, risk history, RTM and architecture rationale. New Part D records M3 design/implementation and points to authentic local source and future CI evidence. The M3 source snapshot is not an authenticated GitHub release and none of the future GitHub actions have been performed.

Team: Thaban Ndlozi (602341); Anele Nkayi (577168); Lebogang Masia (601562). Version history: v1.0 dated 9 Sep 2026; v2.0 dated 26 Sep 2026; current continued v3.0 drafted 12 Oct 2026 and pending team sign-off.

Submission block (fill only after repository governance and review): existing GitHub repository URL = PENDING; assessed M3 release tag/SHA = PENDING (temporary local label m3-rc1-pending-git-tag is not a Git tag); live RTM link = PENDING reconciliation, working continued copy at docs/registers/RTM_CONTINUED.csv; CI/test evidence = .github/workflows/ci.yml and evidence/ with actual run URL PENDING; simulated staging = http://localhost:8081 on the demonstrator machine after Docker startup; API health/version at http://localhost:8082/actuator/health and /api/release; submitted date = PENDING.

D1. Milestone 3 overview and integrated product scope

The original M2 modular monolith, React/TypeScript/Vite client, Spring Boot 4.1.1 / Java 25 server, Flyway PostgreSQL 18.6 persistence and three roles are retained. A requester may submit a categorised validated request, view the request list and an in-app lifecycle timeline. Staff may see all requests, accept a submitted unowned request, and change a request only when authorised as current owner; changes carry optimistic versions and comments. Management may inspect request status totals, category/status breakdowns and accountable actor-aware histories. Browser searches, filters and sorting are provided for visible requests. Refer to README.md, backend/src, frontend/src, docs/m3/m3-baseline-and-scope.md and docs/api.md.

The submitted source is only a candidate for integrated execution: this workspace lacks Java 25/Maven, Docker and dependency downloads, so no full-stack success is asserted. A-005 (approved category/status/overdue business rules), NFR-001 human usability confirmation and GitHub governance remain open. The defined overdue classification is not fabricated.

D2. M2 baseline conformance and controlled evolution

Conformance: modular-monolith modules for requests, audit, reporting, security and frontend still correspond to ADR-M2-ARCH-001. PostgreSQL and Flyway V1 remain the persistence baseline; atomic status/audit writes and after-commit feedback event retain ADR-M2-DES-001. Java 25, Spring Boot 4.1.1, React 19, TypeScript/Vite, Node 24, PostgreSQL 18.6 and container-ready delivery follow ADR-M2-TECH-001, ADR-M2-DEP-001 and ADR-M2-CI-001.

ADR-M3-001 repairs missing version in the M2 prototype status command, aligning it with ADR-M2-DES-002 and returning 409 on stale updates. ADR-M3-002 exposes constrained requester feedback and management actor audit without adding unauthorised data access or a third-party messaging dependency. ADR-M3-003 selects a separated loopback-only Compose simulated staging configuration, not an unapproved cloud provider. Refer to docs/registers/ADR_Change_Log_M3.md for original decision, observed gap, revised decision, consequences and

trace.

D3. Construction, collaboration and secure configuration

backend/ retains the same Spring domain/application/web/persistence modules; frontend/ extends the submitted role UI; PostgreSQL remains schema-controlled by Flyway V1. ChangeStatusCommand includes a mandatory version. RequestService enforces ownership and allowed transition, requires a reason on rejection/resolution/closure, persists actions and audits together, and maps history to role-filtered responses. The controller layer provides requester history, staff details, manager activity and grouped status counts.

The working GitHub Actions definition at .github/workflows/ci.yml uses PR/push quality gates. GitHub main branch protection, actual PR history, issues, two substantive non-author peer approvals and a real CI run have not yet been performed, by team request. GITHUB_AFTER_READY.md gives procedures that preserve the existing repository history and do not manufacture evidence. Credentials are injected via environment. Staging .env.staging is generated locally and git-ignored. Dev/staging Basic authentication is strictly nonproduction; public deployment is blocked pending approved identity and HTTPS controls.

D4. CI, automated verification and static/security controls

The prepared CI pipeline runs Maven verify/JUnit, npm test and TypeScript/Vite compilation, Python limited static/source scanning, TruffleHog verified secret scanning, npm production-dependency audit, a disposable Docker Compose simulated staging stack, live system/API tests, a modest management load exercise and JSON/report artifact upload. Mandatory build/test/check failures block progression. Backend JUnit Surefire XML and JSON staging outputs must be linked from real CI runs; no run links yet exist.

At least five unit/component cases appear in ServiceRequestRulesTest, RequestServiceM3Test and TransitionDecisionTableTest; at least five black-box equivalents/boundaries in CreateRequestValidationTest; at least six planned live API cases and two cross-module E2E/system journeys in scripts/smoke.py. See docs/m3/test-plan-and-case-matrix.md for exact input, oracle, technique, risk, test ID and evidence location. Risk priority is identity/ownership integrity, status/audit consistency and requester privacy. The lightweight local Python syntax/secret-pattern check did pass; see evidence/local-static-check.txt. Do not conflate this with a successful compile or integration run.

D5. Black-box, API/integration, E2E and acceptance

Black-box input design uses equivalence partitioning (valid/nonvalid title/category), boundary analysis (title lengths 1/160/161 and description 4000/4001), a decision/state table for allowed/forbidden status transitions, and negative auth and stale version behaviour. These techniques were selected because they map to real validation, privacy and lifecycle risks. API01-API06 exercise real Spring Security and database-bound HTTP once staged, including expected 401, 403, 400 and 409. E2E01 spans create->list->accept->in progress->resolved->closed->requester audit; E2E02 spans requester create->staff action->manager audit/counts/breakdown. They must be executed on the identified actual staged app before acceptance is claimed. Manual browser role journeys are required to assess NFR-001; the API-only automation cannot establish accessibility or ease of use.

D6. Performance, reliability and staging

PERF-01 targets GET /api/management/summary: as activity grows it repeatedly aggregates persistent request counts and may influence dashboard responsiveness. The proposed workload is 40 authenticated requests with 4 concurrent workers to a dedicated local staging API, collecting p50/p95 response time, error count, throughput and environment/time. A preliminary internal smoke hypothesis is zero errors and p95 below 1 second in that small environment; no stakeholder performance SLO has been approved and **no actual latency/throughput has been measured yet**. See scripts/performance.py, evidence/performance-results.json (generated only after running), and docs/m3/staging-deployment-and-recovery.md.

Staging is a separate Spring profile, database volume and generated credential set. The API release/version endpoint and exact final SHA/tag form the release identity check. A human can demonstrate on http://localhost:8081 (local computer only). Backup/rollback requires a prior verified image/tag, compatible schema and a tested PostgreSQL restore; a recovery drill was not done. The simulation does not prove production-scale use, uptime, third-party availability or secure public access.

D7. Defects, quality evidence and traceability

Continue the original FR-001..010 and NFR-001..004 RTM history with M2 component/ADR trace and M3 acceptance oracles, test IDs, evidence status, defects and residual risks. The supplied ZIP did not contain the actual live editable team RTM, so docs/registers/RTM_CONTINUED.csv is an honest PDF-derived continuation that the team must reconcile with the live original in GitHub; do not overwrite authentic history. Full original acceptance wording is in preserved historical PED v2 Sec 6.

Fixes present in candidate source: DEF-001 inadvertent ownership takeover, DEF-002 missing optimistic request version, DEF-003 missing requester/manager history presentation, DEF-006 missing resolution reasons. They are not marked verified or closed until actual regression evidence exists. DEF-004 (overdue rule) and DEF-005 (human UI acceptance) are open. A passing Java test is narrower than DB transaction proof; a green CI is narrower than business acceptance. See docs/registers/Defect_Register_M3.csv and docs/m3/test-plan-and-case-matrix.md.

D8. Updated registers, AI accountability and residual risks

Original RISK-001..008 and M1/M2 decision records are preserved in the historical PED. Updated M3 risk notes are in docs/registers/Risk_Register_M3.md; high risks include RISK-009 Basic auth on isolated staging, RISK-010 unapproved A-005 business rules, RISK-011 no runtime evidence and RISK-012 inability to simulate production reliability. Technical debts DEBT-001..007 include authentication, final policy, client-side scalability, no push notifications and recovery drills. The AI usage register documents the AI-assisted preparation and expressly records unavailable verification; students must own correctness, review fixes and reject unsupported evidence. See docs/registers/Technical_Debt_Register_M3.csv, AI_Usage_Register_M3.md and ADR_Change_Log_M3.md.

D9. Release-quality review and final judgement

Classification on 12 October 2026: **NOT READY** for formal M3 evidence sign-off, because Maven/JUnit, npm/Vitest/build, live DB/API/E2E, performance results, GitHub CI/reviews, final release tag and stakeholder acceptance are unverified or unapproved in this environment. Source-level design continuity and the limited local static check are evidenced. Consult docs/m3/release-evidence-summary.md for gate-by-gate status and remediation. Never relabel until the exact commit passes required CI, staging E2E, measured load and human acceptance with recorded defects/risks.

Residual risk persists after passing tests: final category/status/overdue decisions, staging-vs-production identity/TLS/backup, insufficient representative load, and client-side filters. Real production release would require materially stronger hardening, stakeholder rules, identity verification, backups/restore and operational checks. A M3

student-capacity release candidate can become CONDITIONALLY READY when core evidence actually passes and nonblocking constraints are agreed.

D10. Traceability, demo, submission and defence index

Pedagogic trace pattern: STK-01/02/03/04 -> original FR/NFR and AC (historical PED Sec 6) -> ADR-M2/ADR-M3 -> implementation class/endpoint -> identified testcase -> actual CI/JUnit/JSON result -> defect/risk -> release judgement. Example FR-010: manager accountability AC-FR-010 -> ADR-M2-DES-001 -> RequestAudit/ManagementController/AuditView -> E2E02 expected actor/action/time -> real staging output pending -> RISK-011 -> NOT READY. Example FR-007: controlled transitions AC-FR-007 -> ADR-M2-DES-002 + ADR-M3-001 -> RequestService/Policy/version -> decision-table JUnit and API04 invalid 409 -> execution pending -> DEF-002 -> release gate pending.

For the live team demo show the actual final Git tag and full commit, review discussions with two non-author approvals, CI results, staging health/release identity, role workflows, negative access, API/system tests, actual load results, quality gates and risk rationale. Every student must explain one complete trace, change, failure and release judgement, not only a scripted role. Read docs/m3/team-demo-and-individual-defence.md and docs/m3/m3-submission-checklist.md. Formal Moodle submission normally consists of the *continued PED v3.0 PDF*, actual GitHub URL, assessed RC commit/tag and staging/demo information, not a detached source ZIP.